INSTITUTO POLITÉCNICO NACIONAL

UNIDAD PROFESIONAL INTERDISCIPLINARIA DE INGENIERÍA Y CIENCIAS SOCIALES Y ADMINISTRATIVAS

REPORTES 2DO DEPARTAMENTAL

ABSTRACCIÓN Y USO DE DATOS

CIENCIAS DE LA INFORMÁTICA

PROFESOR: YVENTZ ENTZANA GARDUÑO

3CM31

- GARDUÑO MUÑOZ GANDHI
- OSORIO GUADARRAMA GERARDO

Contenido

Ejercicio 16 - Ordenamiento Burbuja (PE y POO) 3

Ejercicio 17 - Ordenamiento Merge Sort (PE y POO) 7

Ejercicio 18 - Ordenamiento Quick Sort (PE y POO) 11

Ejercicio 19 - Ordenamiento de manera indirecta (Burbuja) 16

Ejercicio 20 - Ordenamiento de manera indirecta (Merge Sort) 21

Ejercicio 21 - Ordenamiento de manera indirecta (Quick Sort) con Ingreso de Datos 27

Ejercicio 1 - Punteros para el manejo indirecto con clases y estructuras 33

Ejercicio 3 - Pila estática por medio de un arreglo (PE y POO) 39

Ejercicio 4 - Cola estática por medio de un arreglo (PE y POO) 43

Ejercicio 5 - Lista estática por medio de un arreglo (PE y POO) 48

Ejercicio 6 - Pila dinámica por medio de punteros al mismo tipo de dato (PE y POO) 54

Ejercicio 7 - Cola dinámica por medio de punteros al mismo tipo de dato (PE y POO) 61

Ejercicio 7 bis - Manejo de una lista de manera dinámica por medio de punteros al mismo tipo de dato (PE y POO) 68

Ejercicio 8 - Pila dinámica por medio de las librerías (PE y POO) .. 78

Ejercicio 9 - Cola dinámica por medio de las librerías (PE y POO) .. 84

Ejercicio 10 - Manejo de una lista de manera dinámica por medio de las librerías (PE y POO) 91

Ejercicio 16 - Ordenamiento Burbuja (PE y POO)

Objetivo del código

Implementar el algoritmo de ordenamiento burbuja para caracteres, enteros y un nuevo tipo de dato abstracto (Persona), aplicando Programación Estructurada (PE) y Programación Orientada a Objetos (POO). El programa pide los datos por consola antes de ordenarlos. Las personas se ordenan por edad.

Código PE

Ejercicio16_PE.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

struct Persona {
    string nombre;
    int edad;
};

// Criterio de comparación para cada tipo de dato
bool mayor(int a, int b)         { return a > b; }
bool mayor(char a, char b)       { return a > b; }
bool mayor(Persona a, Persona b) { return a.edad > b.edad; }

// Burbuja genérico: sirve para int, char y Persona
template <typename T>
    void burbuja(T arr[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (mayor(arr[j], arr[j + 1])) {
                T aux = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = aux;
    }
}

```

```cpp
int main() {
    const int N = 2;
    int enteros[N];
    char caracteres[N];
    Persona personas[N];

    cout << "--- Captura de Enteros ---\n";
    for (int i = 0; i < N; i++) {
        cout << "Ingresa entero " << i + 1 << ": ";
        cin >> enteros[i];
    }
    cout << "\n--- Captura de Caracteres ---\n";
    for (int i = 0; i < N; i++) {
        cout << "Ingresa caracter " << i + 1 << ": ";
        cin >> caracteres[i];
    }
    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < N; i++) {
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> personas[i].nombre;
        cout << "Edad de persona " << i + 1 << ": ";
        cin >> personas[i].edad;
    }

    burbuja(enteros, N);
    burbuja(caracteres, N);
    burbuja(personas, N);

    cout << "\nResultados PE\n";
    cout << "Enteros ordenados: ";
    for (int i = 0; i < N; i++) cout << enteros[i] << " ";
    cout << "\nCaracteres ordenados: ";
    for (int i = 0; i < N; i++) cout << caracteres[i] << " ";
    cout << "\nPersonas ordenadas por edad: ";
    for (int i = 0; i < N; i++) cout << personas[i].nombre << " (" << personas[i].edad << ") ";
    cout << endl;
    return 0;
}
```

Salida

```
D:\Programas\Nueva carpeta\
--- Captura de Enteros ---
Ingresa entero 1: 23
Ingresa entero 2: 56

--- Captura de Caracteres ---
Ingresa caracter 1: g
Ingresa caracter 2: e

--- Captura de Personas ---
Nombre de persona 1: Clark
Edad de persona 1: 22
Nombre de persona 2: Lex
Edad de persona 2: 25

Resultados PE
Enteros ordenados: 23 56
Caracteres ordenados: e g
Personas ordenadas por edad: Clark (22) Lex (25)


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Código POO

Ejercicio16_POO.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

class Persona {
private:
    string nombre;
    int edad;
public:
    void setNombre(string n) { nombre = n; }
    void setEdad(int e)      { edad = e; }
    string getNombre()       { return nombre; }
    int getEdad()            { return edad; }
};

class OrdenadorBurbuja {
private:
    // Criterio de comparación para cada tipo de dato
    bool mayor(int a, int b)         { return a > b; }
    bool mayor(char a, char b)       { return a > b; }
    bool mayor(Persona a, Persona b) { return a.getEdad() > b.getEdad(); }
public:
        template <typename T>
        void ordenar(T arr[], int n) {
            for (int i = 0; i < n - 1; i++)
                for (int j = 0; j < n - 1 - i; j++)
                    if (mayor(arr[j], arr[j + 1])) {
                        T aux = arr[j];
                        arr[j] = arr[j + 1];
                        arr[j + 1] = aux;
            }
        }
};

```

```cpp
int main() {
    const int N = 2;
    int enteros[N];
    char caracteres[N];
    Persona personas[N];
    OrdenadorBurbuja ordenador;

    cout << "--- Captura de Enteros ---\n";
    for (int i = 0; i < N; i++) {
        cout << "Ingresa entero " << i + 1 << ": ";
        cin >> enteros[i];
    }
    cout << "\n--- Captura de Caracteres ---\n";
    for (int i = 0; i < N; i++) {
        cout << "Ingresa caracter " << i + 1 << ": ";
        cin >> caracteres[i];
    }
    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < N; i++) {
        string n;
        int e;
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> n;
        cout << "Edad de persona " << i + 1 << ": ";
        cin >> e;
        personas[i].setNombre(n);
        personas[i].setEdad(e);
    }

    ordenador.ordenar(enteros, N);
    ordenador.ordenar(caracteres, N);
    ordenador.ordenar(personas, N);

    cout << "\nResultados POO\n";
    cout << "Enteros ordenados: ";
    for (int i = 0; i < N; i++) cout << enteros[i] << " ";
    cout << "\nCaracteres ordenados: ";
    for (int i = 0; i < N; i++) cout << caracteres[i] << " ";
    cout << "\nPersonas ordenadas por edad: ";
    for (int i = 0; i < N; i++)
        cout << personas[i].getNombre() << " (" << personas[i].getEdad() << ") ";
    cout << endl;
    return 0;
}
```

Salida

```
D:\Programas\Nueva carpeta\
--- Captura de Enteros ---
Ingresa entero 1: 37
Ingresa entero 2: 73

--- Captura de Caracteres ---
Ingresa caracter 1: k
Ingresa caracter 2: g

--- Captura de Personas ---
Nombre de persona 1: Luke
Edad de persona 1: 37
Nombre de persona 2: Jake
Edad de persona 2: 34

Resultados POO
Enteros ordenados: 37 73
Caracteres ordenados: g k
Personas ordenadas por edad: Jake (34) Luke (37)


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Explicación del Código

- Ingreso de datos interactivo: en ambas versiones se utilizan ciclos for y cin >> para pedir al usuario los enteros, los caracteres y las personas antes de ejecutar la lógica de ordenamiento.

- Enfoque PE: los datos de la persona se guardan directamente en los campos de la estructura (personas[i].nombre y personas[i].edad). La función burbuja() es global.

- Enfoque POO: los datos de la persona se guardan primero en variables temporales (nom y e) y después se asignan con los métodos setNombre() y setEdad(), de modo que los atributos permanecen privados (encapsulamiento). La clase OrdenadorBurbuja contiene el algoritmo y los criterios de comparación como métodos privados, y expone un único método público ordenar().

- Algoritmo burbuja: compara cada par de elementos vecinos (arr[j] y arr[j + 1]) y, si el primero es mayor, los intercambia usando una variable auxiliar. El ciclo exterior repite el recorrido y el ciclo interior se acorta (n - 1 - i), porque después de cada pasada el elemento más grande ya quedó en su posición final.

- Un solo algoritmo para tres tipos de dato: template <typename T> permite usar el mismo código con int, char y Persona. La función mayor() está sobrecargada: compara valores en enteros y caracteres, y compara la edad en Persona.

- • Lo que imprime: en la versión PE, los enteros 23 y 56 ya estaban en orden y se conservan, los caracteres g y e quedan como e g, y las personas Clark (22) y Lex (25) también conservan su orden. En la versión POO, los enteros 37 y 73 se conservan, los caracteres k y g quedan como g k, y las personas Luke (37) y Jake (34) se intercambian para quedar Jake (34) antes que Luke (37).

- Errores y Soluciones: no hubo errores.

Ejercicio 17 - Ordenamiento Merge Sort (PE y POO)

Objetivo del código

Implementar el algoritmo de ordenamiento por mezcla (Merge Sort) para caracteres, enteros y un nuevo tipo de dato abstracto (Persona), aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). El programa permite la captura dinámica de los datos por parte del usuario mediante la consola antes de realizar el ordenamiento por el método de “divide y vencerás”.

Código PE

Ejercicio17_PE.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

struct Persona {
    string nombre;
    int edad;
};

bool mayor(int a, int b)         { return a > b; }
bool mayor(char a, char b)       { return a > b; }
bool mayor(Persona a, Persona b) { return a.edad > b.edad; }

template <typename T>
    void mezclar(T arr[], int ini, int med, int fin) {
    T aux[MAX];
    int i = ini, j = med + 1, k = 0;
    while (i <= med && j <= fin) {
        if (mayor(arr[i], arr[j])) aux[k++] = arr[j++];
        else                       aux[k++] = arr[i++];
    }
    while (i <= med) aux[k++] = arr[i++];
    while (j <= fin) aux[k++] = arr[j++];
    for (k = 0; k <= fin - ini; k++) arr[ini + k] = aux[k];
}

    template <typename T>
        void mergeSort(T arr[], int ini, int fin) {
        if (ini >= fin) return;
        int med = (ini + fin) / 2;
        mergeSort(arr, ini, med);
        mergeSort(arr, med + 1, fin);
        mezclar(arr, ini, med, fin);
    }

    int main() {
        int n;
        int enteros[MAX];
        char caracteres[MAX];
        Persona personas[MAX];

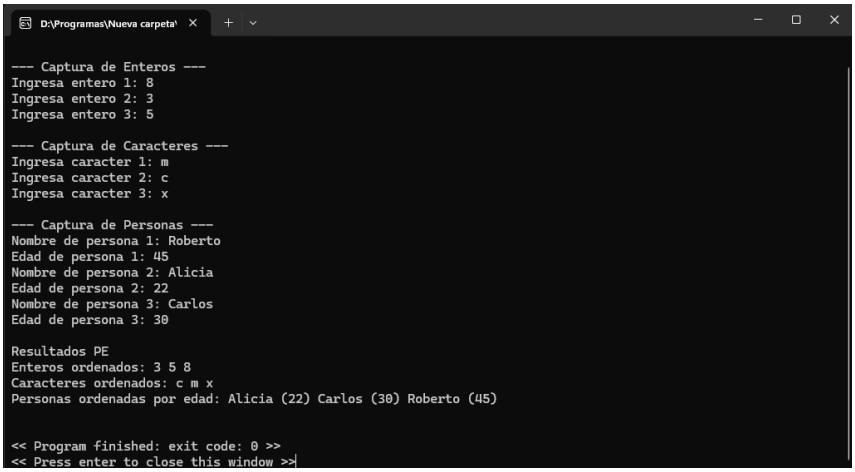
        cout << "Cantidad de elementos (max 100): ";
        cin >> n;

        cout << "\n--- Captura de Enteros ---\n";
        for (int i = 0; i < n; i++) {
            cout << "Ingresa entero " << i + 1 << ": ";
            cin >> enteros[i];
        }
        cout << "\n--- Captura de Caracteres ---\n";
        for (int i = 0; i < n; i++) {
            cout << "Ingresa caracter " << i + 1 << ": ";
            cin >> caracteres[i];
        }
        cout << "\n--- Captura de Personas ---\n";
        for (int i = 0; i < n; i++) {
            cout << "Nombre de persona " << i + 1 << ": ";
            cin >> personas[i].nombre;
            cout << "Edad de persona " << i + 1 << ": ";
            cin >> personas[i].edad;
        }

        mergeSort(enteros, 0, n - 1);
        mergeSort(caracteres, 0, n - 1);
        mergeSort(personas, 0, n - 1);

        cout << "\nResultados PE\n";
        cout << "Enteros ordenados: ";
        for (int i = 0; i < n; i++) cout << enteros[i] << " ";
        cout << "\nCaracteres ordenados: ";
        for (int i = 0; i < n; i++) cout << caracteres[i] << " ";
        cout << "\nPersonas ordenadas por edad: ";
        for (int i = 0; i < n; i++) cout << personas[i].nombre << " (" << personas[i].edad << ") ";
        cout << endl;
        return 0;
    }
```

Salida



Código POO

Ejercicio17_POO.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

class Persona {
private:
    string nombre;
    int edad;
public:
    void setNombre(string n) { nombre = n; }
    void setEdad(int e)      { edad = e; }
    string getNombre()       { return nombre; }
    int getEdad()            { return edad; }
};

class OrdenadorMerge {
private:
    bool mayor(int a, int b)         { return a > b; }
    bool mayor(char a, char b)       { return a > b; }
    bool mayor(Persona a, Persona b) { return a.getEdad() > b.getEdad(); }

    template <typename T>
    void mezclar(T arr[], int ini, int med, int fin) {
        T aux[MAX];
        int i = ini, j = med + 1, k = 0;
        while (i <= med && j <= fin) {
            if (mayor(arr[i], arr[j])) aux[k++] = arr[j++];
            else                       aux[k++] = arr[i++];
        }
        while (i <= med) aux[k++] = arr[i++];
        while (j <= fin) aux[k++] = arr[j++];
        for (k = 0; k <= fin - ini; k++) arr[ini + k] = aux[k];
    }

    template <typename T>
    void mergeSort(T arr[], int ini, int fin) {
        if (ini >= fin) return;
        int med = (ini + fin) / 2;
        mergeSort(arr, ini, med);
        mergeSort(arr, med + 1, fin);
        mezclar(arr, ini, med, fin);
    }

public:
    template <typename T>
    void ordenar(T arr[], int n) { mergeSort(arr, 0, n - 1); }
};
```

```cpp
int main() {
    int n;
    int enteros[MAX];
    char caracteres[MAX];
    Persona personas[MAX];
    OrdenadorMerge ordenador;

    cout << "Cantidad de elementos (max 100): ";
    cin >> n;

    cout << "\n--- Captura de Enteros ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa entero " << i + 1 << ": ";
        cin >> enteros[i];
    }
    cout << "\n--- Captura de Caracteres ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa caracter " << i + 1 << ": ";
        cin >> caracteres[i];
    }
    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < n; i++) {
        string nom;
        int e;
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> nom;
        cout << "Edad de persona " << i + 1 << ": ";
        cin >> e;
        personas[i].setNombre(nom);
        personas[i].setEdad(e);
    }

    ordenador.ordenar(enteros, n);
    ordenador.ordenar(caracteres, n);
    ordenador.ordenar(personas, n);

    cout << "\nResultados POO\n";
    cout << "Enteros ordenados: ";
    for (int i = 0; i < n; i++) cout << enteros[i] << " ";
    cout << "\nCaracteres ordenados: ";
    for (int i = 0; i < n; i++) cout << caracteres[i] << " ";
    cout << "\nPersonas ordenadas por edad: ";
    for (int i = 0; i < n; i++)
        cout << personas[i].getNombre() << " (" << personas[i].getEdad() << ") ";
    cout << endl;
    return 0;
}
```

Salida

```
D:\Programas\Nueva carpeta\
Ingresa entero 3: 5

--- Captura de Caracteres ---
Ingresa caracter 1: m
Ingresa caracter 2: c
Ingresa caracter 3: x

--- Captura de Personas ---
Nombre de persona 1: Roberto
Edad de persona 1: 45
Nombre de persona 2: Alicia
Edad de persona 2: 22
Nombre de persona 3: Carlos
Edad de persona 3: 30

Resultados POO
Enteros ordenados: 3 5 8
Caracteres ordenados: c m x
Personas ordenadas por edad: Alicia (22) Carlos (30) Roberto (45)


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Explicación del Código

- Ingreso de datos dinámico: primero se pide la cantidad de elementos (n) y después se capturan con ciclos for y cin >> los enteros, los caracteres y las personas.
- Enfoque PE: los datos de la persona se guardan directamente en los campos de la estructura (personas[i].nombre y personas[i].edad). Las funciones mergeSort() y mezclar() son globales.
- Enfoque POO: los datos de la persona se guardan primero en variables temporales (nom y e) y después se asignan con los métodos setNombre() y setEdad(), de modo que los atributos permanecen privados (encapsulamiento). La clase OrdenadorMerge contiene el algoritmo y los criterios de comparación como métodos privados, y expone un único método público ordenar().
- Divide y vencerás: mergeSort() divide el arreglo por la mitad de forma recursiva hasta que cada parte tiene un solo elemento (caso base: ini >= fin).
- Mezcla: mezclar() combina dos mitades ya ordenadas. Compara el primer elemento de cada mitad, copia el menor a un arreglo auxiliar y al final copia el resultado de vuelta al arreglo original.
- Un solo algoritmo para tres tipos de dato: template <typename T> permite usar el mismo código con int, char y Persona. La función mayor() está sobrecargada: compara valores en enteros y caracteres, y compara la edad en Persona.
- Lo que imprime: con los datos de ejemplo, los enteros 8, 3 y 5 quedan como 3 5 8, los caracteres m, c y x quedan como c m x, y las personas quedan ordenadas por edad: Alicia (22), Carlos (30) y Roberto (45). El resultado es el mismo en PE y POO.
- Errores y Soluciones: no hubo errores.

Ejercicio 18 - Ordenamiento Quick Sort (PE y POO)

Objetivo del código

Implementar el algoritmo de ordenamiento rápido (Quick Sort) para caracteres, enteros y un nuevo tipo de dato abstracto (Persona), aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). El programa permite la captura dinámica de los datos por parte del usuario mediante la consola antes de realizar el ordenamiento, eligiendo un pivote y dividiendo el arreglo en elementos menores y mayores que él.

Código PE

Ejercicio18_PE.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

struct Persona {
    string nombre;
    int edad;
};

bool mayor(int a, int b)         { return a > b; }
bool mayor(char a, char b)       { return a > b; }
bool mayor(Persona a, Persona b) { return a.edad > b.edad; }

template <typename T>
    int particion(T arr[], int ini, int fin) {
    T pivote = arr[fin];
    int i = ini - 1;
    for (int j = ini; j < fin; j++) {
        if (mayor(pivote, arr[j])) {
            i++;
            T aux = arr[i];
            arr[i] = arr[j];
            arr[j] = aux;
        }
    }
    T aux = arr[i + 1];
    arr[i + 1] = arr[fin];
    arr[fin] = aux;
    return i + 1;
}

    template <typename T>
        void quickSort(T arr[], int ini, int fin) {
        if (ini >= fin) return;
        int p = particion(arr, ini, fin);
        quickSort(arr, ini, p - 1);
        quickSort(arr, p + 1, fin);
    }
```

```cpp
int main() {
    int n;
    int enteros[MAX];
    char caracteres[MAX];
    Persona personas[MAX];

    cout << "Cantidad de elementos (max 100): ";
    cin >> n;

    cout << "\n--- Captura de Enteros ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa entero " << i + 1 << ": ";
        cin >> enteros[i];
    }
    cout << "\n--- Captura de Caracteres ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa caracter " << i + 1 << ": ";
        cin >> caracteres[i];
    }
    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> personas[i].nombre;
        cout << "Edad de persona " << i + 1 << ": ";
        cin >> personas[i].edad;
    }

    quickSort(enteros, 0, n - 1);
    quickSort(caracteres, 0, n - 1);
    quickSort(personas, 0, n - 1);

    cout << "\nResultados PE\n";
    cout << "Enteros ordenados: ";
    for (int i = 0; i < n; i++) cout << enteros[i] << " ";
    cout << "\nCaracteres ordenados: ";
    for (int i = 0; i < n; i++) cout << caracteres[i] << " ";
    cout << "\nPersonas ordenadas por edad: ";
    for (int i = 0; i < n; i++) cout << personas[i].nombre << " (" << personas[i].edad << ") ";
    cout << endl;
    return 0;
}
```

Salida

```
D:\Programas\Nueva carpeta\
Ingresa entero 1: 9
Ingresa entero 2: 2
Ingresa entero 3: 7
Ingresa entero 4: 4

--- Captura de Caracteres ---
Ingresa caracter 1: q
Ingresa caracter 2: d
Ingresa caracter 3: z
Ingresa caracter 4: a

--- Captura de Personas ---
Nombre de persona 1: Roberto
Edad de persona 1: 45
Nombre de persona 2: Alicia
Edad de persona 2: 22
Nombre de persona 3: Carlos
Edad de persona 3: 30
Nombre de persona 4: Diana
Edad de persona 4: 19

Resultados PE
Enteros ordenados: 2 4 7 9
Caracteres ordenados: a d q z
Personas ordenadas por edad: Diana (19) Alicia (22) Carlos (30) Roberto (45)


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Código POO

Ejercicio18_POO.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

class Persona {
private:
    string nombre;
    int edad;
public:
    void setNombre(string n) { nombre = n; }
    void setEdad(int e)      { edad = e; }
    string getNombre()       { return nombre; }
    int getEdad()            { return edad; }
};

class OrdenadorQuick {
private:
    bool mayor(int a, int b)         { return a > b; }
    bool mayor(char a, char b)       { return a > b; }
    bool mayor(Persona a, Persona b) { return a.getEdad() > b.getEdad(); }

    template <typename T>
    int particion(T arr[], int ini, int fin) {
        T pivote = arr[fin];
        int i = ini - 1;
        for (int j = ini; j < fin; j++) {
            if (mayor(pivote, arr[j])) {
                i++;
                T aux = arr[i];
                arr[i] = arr[j];
                arr[j] = aux;
            }
        }
        T aux = arr[i + 1];
        arr[i + 1] = arr[fin];
        arr[fin] = aux;
        return i + 1;
    }

    template <typename T>
    void quickSort(T arr[], int ini, int fin) {
        if (ini >= fin) return;
        int p = particion(arr, ini, fin);
        quickSort(arr, ini, p - 1);
        quickSort(arr, p + 1, fin);
    }

public:
    template <typename T>
    void ordenar(T arr[], int n) { quickSort(arr, 0, n - 1); }
};
```

```cpp
int main() {
    int n;
    int enteros[MAX];
    char caracteres[MAX];
    Persona personas[MAX];
    OrdenadorQuick ordenador;

    cout << "Cantidad de elementos (max 100): ";
    cin >> n;

    cout << "\n--- Captura de Enteros ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa entero " << i + 1 << ": ";
        cin >> enteros[i];
    }
    cout << "\n--- Captura de Caracteres ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa caracter " << i + 1 << ": ";
        cin >> caracteres[i];
    }
    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < n; i++) {
        string nom;
        int e;
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> nom;
        cout << "Edad de persona " << i + 1 << ": ";
        cin >> e;
        personas[i].setNombre(nom);
        personas[i].setEdad(e);
    }

    ordenador.ordenar(enteros, n);
    ordenador.ordenar(caracteres, n);
    ordenador.ordenar(personas, n);

    cout << "\nResultados POO\n";
    cout << "Enteros ordenados: ";
    for (int i = 0; i < n; i++) cout << enteros[i] << " ";
    cout << "\nCaracteres ordenados: ";
    for (int i = 0; i < n; i++) cout << caracteres[i] << " ";
    cout << "\nPersonas ordenadas por edad: ";
    for (int i = 0; i < n; i++)
        cout << personas[i].getNombre() << " (" << personas[i].getEdad() << ") ";
    cout << endl;
    return 0;
}
```

Salida

```
D:\Programas\Nueva carpeta\

--- Captura de Enteros ---
Ingresa entero 1: 9
Ingresa entero 2: 2
Ingresa entero 3: 7
Ingresa entero 4: 4

--- Captura de Caracteres ---
Ingresa caracter 1: q
Ingresa caracter 2: d
Ingresa caracter 3: z
Ingresa caracter 4: a

--- Captura de Personas ---
Nombre de persona 1: Roberto
Edad de persona 1: 45
Nombre de persona 2: Alicia
Edad de persona 2: 22
Nombre de persona 3: Carlos
Edad de persona 3: 30
Nombre de persona 4: Diana
Edad de persona 4: 19

Resultados POO
Enteros ordenados: 2 4 7 9
Caracteres ordenados: a d q z
Personas ordenadas por edad: Diana (19) Alicia (22) Carlos (30) Roberto (45)


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Explicación del Código

- Ingreso de datos dinámico: primero se pide la cantidad de elementos (n) y después se capturan con ciclos for y cin >> los enteros, los caracteres y las personas.
- Enfoque PE: los datos de la persona se guardan directamente en los campos de la estructura (personas[i].nombre y personas[i].edad). Las funciones quickSort() y particion() son globales.
- Enfoque POO: los datos de la persona se guardan primero en variables temporales (nom y e) y después se asignan con los métodos setNombre() y setEdad(), de modo que los atributos permanecen privados (encapsulamiento). La clase OrdenadorQuick contiene el algoritmo y los criterios de comparación como métodos privados, y expone un único método público ordenar().
- Divide y vencerás: quickSort() elige un pivote (el último elemento), lo coloca en su posición final con particion() y se llama de forma recursiva sobre la parte izquierda (menores) y la parte derecha (mayores). El caso base es ini >= fin.
- Partición: particion() recorre el arreglo y mueve al inicio los elementos menores que el pivote mediante intercambios con una variable auxiliar. Al final, el pivote se intercambia a la posición que le corresponde y esa posición se devuelve.
- Un solo algoritmo para tres tipos de dato: template <typename T> permite usar el mismo código con int, char y Persona. La función mayor() está sobrecargada: compara valores en enteros y caracteres, y compara la edad en Persona.
- Lo que imprime: con los datos de ejemplo, los enteros 9, 2, 7 y 4 quedan como 2 4 7 9, los caracteres q, d, z y a quedan como a d q z, y las personas quedan ordenadas por edad: Diana (19), Alicia (22), Carlos (30) y Roberto (45). El resultado es el mismo en PE y POO; solo cambia el encabezado “Resultados PE” / “Resultados POO”.
- Errores y Soluciones: no hubo errores.

Ejercicio 19 - Ordenamiento de manera indirecta (Burbuja)

Objetivo del código

Implementar el algoritmo de ordenamiento burbuja de manera indirecta para caracteres, enteros y un nuevo tipo de dato abstracto (Persona), aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). En lugar de mover los datos, se ordena un arreglo de punteros que apuntan a ellos, por lo que el arreglo original permanece sin cambios. El programa permite la captura dinámica de los datos por consola antes de ordenarlos.

Código PE

Ejercicio19_PE.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

struct Persona {
    string nombre;
    int edad;
};

bool mayor(int a, int b)         { return a > b; }
bool mayor(char a, char b)       { return a > b; }
bool mayor(Persona a, Persona b) { return a.edad > b.edad; }

template <typename T>
    void burbujaIndirecta(T* ptr[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - 1 - i; j++)
            if (mayor(*ptr[j], *ptr[j + 1])) {
                T* aux = ptr[j];
                ptr[j] = ptr[j + 1];
                ptr[j + 1] = aux;
    }
}

    int main() {
        int n;
        int enteros[MAX];
        char caracteres[MAX];
        Persona personas[MAX];
        int* ptrEnteros[MAX];
        char* ptrCaracteres[MAX];
        Persona* ptrPersonas[MAX];

        cout << "Cantidad de elementos (max 100): ";
        cin >> n;

        cout << "\n--- Captura de Enteros ---\n";
        for (int i = 0; i < n; i++) {
            cout << "Ingresa entero " << i + 1 << ": ";
            cin >> enteros[i];
            ptrEnteros[i] = &enteros[i];
        }
        cout << "\n--- Captura de Caracteres ---\n";
        for (int i = 0; i < n; i++) {
            cout << "Ingresa caracter " << i + 1 << ": ";
            cin >> caracteres[i];
            ptrCaracteres[i] = &caracteres[i];
        }
```

```
    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> personas[i].nombre;
        cout << "Edad de persona " << i + 1 << ": ";
        cin >> personas[i].edad;
        ptrPersonas[i] = &personas[i];
    }

    burbujaIndirecta(ptrEnteros, n);
    burbujaIndirecta(ptrCaracteres, n);
    burbujaIndirecta(ptrPersonas, n);

    cout << "\nResultados PE\n";
    cout << "Enteros originales: ";
    for (int i = 0; i < n; i++) cout << enteros[i] << " ";
    cout << "\nEnteros ordenados (indirecto): ";
    for (int i = 0; i < n; i++) cout << *ptrEnteros[i] << " ";

    cout << "\nCaracteres originales: ";
    for (int i = 0; i < n; i++) cout << caracteres[i] << " ";
    cout << "\nCaracteres ordenados (indirecto): ";
    for (int i = 0; i < n; i++) cout << *ptrCaracteres[i] << " ";

    cout << "\nPersonas originales: ";
    for (int i = 0; i < n; i++) cout << personas[i].nombre << " (" << personas[i].edad << ") ";
    cout << "\nPersonas ordenadas por edad (indirecto): ";
    for (int i = 0; i < n; i++) cout << ptrPersonas[i]->nombre << " (" << ptrPersonas[i]->edad << ") ";
    cout << endl;
    return 0;
}
```

Salida

```
D:\Programas\Nueva carpeta\

Ingresa entero 3: 7

--- Captura de Caracteres ---
Ingresa caracter 1: q
Ingresa caracter 2: d
Ingresa caracter 3: a

--- Captura de Personas ---
Nombre de persona 1: Roberto
Edad de persona 1: 45
Nombre de persona 2: Alicia
Edad de persona 2: 22
Nombre de persona 3: Carlos
Edad de persona 3: 30

Resultados PE
Enteros originales: 9 2 7
Enteros ordenados (indirecto): 2 7 9
Caracteres originales: q d a
Caracteres ordenados (indirecto): a d q
Personas originales: Roberto (45) Alicia (22) Carlos (30)
Personas ordenadas por edad (indirecto): Alicia (22) Carlos (30) Roberto (45)


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Código POO

Ejercicio19_POO.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

class Persona {
private:
    string nombre;
    int edad;
public:
    void setNombre(string n) { nombre = n; }
    void setEdad(int e)      { edad = e; }
    string getNombre()       { return nombre; }
    int getEdad()            { return edad; }
};

class OrdenadorIndirecto {
private:
    bool mayor(int a, int b)         { return a > b; }
    bool mayor(char a, char b)       { return a > b; }
    bool mayor(Persona a, Persona b) { return a.getEdad() > b.getEdad(); }

public:
    template <typename T>
    void ordenar(T* ptr[], int n) {
        for (int i = 0; i < n - 1; i++)
            for (int j = 0; j < n - 1 - i; j++)
                if (mayor(*ptr[j], *ptr[j + 1])) {
                    T* aux = ptr[j];
                    ptr[j] = ptr[j + 1];
                    ptr[j + 1] = aux;
                }
    }
};

int main() {
    int n;
    int enteros[MAX];
    char caracteres[MAX];
    Persona personas[MAX];
    int* ptrEnteros[MAX];
    char* ptrCaracteres[MAX];
    Persona* ptrPersonas[MAX];
    OrdenadorIndirecto ordenador;

    cout << "Cantidad de elementos (max 100): ";
    cin >> n;

```

```
    cout << "\n--- Captura de Enteros ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa entero " << i + 1 << ": ";
        cin >> enteros[i];
        ptrEnteros[i] = &enteros[i];
    }
    cout << "\n--- Captura de Caracteres ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa caracter " << i + 1 << ": ";
        cin >> caracteres[i];
        ptrCaracteres[i] = &caracteres[i];
    }
    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < n; i++) {
        string nom;
        int e;
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> nom;
        cout << "Edad de persona " << i + 1 << ": ";
        cin >> e;
        personas[i].setNombre(nom);
        personas[i].setEdad(e);
        ptrPersonas[i] = &personas[i];
    }

    ordenador.ordenar(ptrEnteros, n);
    ordenador.ordenar(ptrCaracteres, n);
    ordenador.ordenar(ptrPersonas, n);

    cout << "\nResultados POO\n";
    cout << "Enteros originales: ";
    for (int i = 0; i < n; i++) cout << enteros[i] << " ";
    cout << "\nEnteros ordenados (indirecto): ";
    for (int i = 0; i < n; i++) cout << *ptrEnteros[i] << " ";

    cout << "\nCaracteres originales: ";
    for (int i = 0; i < n; i++) cout << caracteres[i] << " ";
    cout << "\nCaracteres ordenados (indirecto): ";
    for (int i = 0; i < n; i++) cout << *ptrCaracteres[i] << " ";

    cout << "\nPersonas originales: ";
    for (int i = 0; i < n; i++)
        cout << personas[i].getNombre() << " (" << personas[i].getEdad() << ") ";
    cout << "\nPersonas ordenadas por edad (indirecto): ";
    for (int i = 0; i < n; i++)
        cout << ptrPersonas[i]->getNombre() << " (" << ptrPersonas[i]->getEdad() << ") ";
    cout << endl;
    return 0;
}
```

Salida

```
D:\Programas\Nueva carpeta\
Cantidad de elementos (max 100): 3

--- Captura de Enteros ---
Ingresa entero 1: 9
Ingresa entero 2: 2
Ingresa entero 3: 7

--- Captura de Caracteres ---
Ingresa caracter 1: q
Ingresa caracter 2: d
Ingresa caracter 3: a

--- Captura de Personas ---
Nombre de persona 1: Roberto
Edad de persona 1: 45
Nombre de persona 2: Alicia
Edad de persona 2: 22
Nombre de persona 3: Carlos
Edad de persona 3: 30

Resultados POO
Enteros originales: 9 2 7
Enteros ordenados (indirecto): 2 7 9
Caracteres originales: q d a
Caracteres ordenados (indirecto): a d q
Personas originales: Roberto (45) Alicia (22) Carlos (30)
Personas ordenadas por edad (indirecto): Alicia (22) Carlos (30) Roberto (45)


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Explicación del Código

- Ingreso de datos dinámico: primero se pide la cantidad de elementos (n) y después se capturan con ciclos for y cin >> los enteros, los caracteres y las personas. Al capturar cada dato se guarda su dirección de memoria (&) en el arreglo de punteros correspondiente.

- Ordenamiento indirecto: el algoritmo burbuja no mueve los datos; intercambia los punteros del arreglo auxiliar. Para comparar se accede al valor apuntado con el operador de desreferencia (*ptr[j]), y al terminar los punteros quedan apuntando a los datos en orden.

- Arreglo original intacto: como solo se reacomodan los punteros, los arreglos enteros, caracteres y personas conservan su orden de captura. Por eso el programa imprime primero el arreglo original y después los datos recorridos a través de los punteros ya ordenados.

- Acceso a los campos de Persona: en el arreglo de punteros se usa el operador flecha (ptrPersonas[i]->nombre en PE, ptrPersonas[i]->getNombre() en POO) para leer los datos de la persona a la que apunta cada puntero.

- Enfoque PE: los datos de la persona se guardan directamente en los campos de la estructura (personas[i].nombre y personas[i].edad) y la función burbujaIndirecta() es global.

- Enfoque POO: los datos de la persona se guardan primero en variables temporales (nom y e) y después se asignan con los métodos setNombre() y setEdad(), de modo que los atributos permanecen privados (encapsulamiento). La clase OrdenadorIndirecto contiene los criterios de comparación como métodos privados y expone un único método público ordenar().

- Un solo algoritmo para tres tipos de dato: template <typename T> permite usar el mismo código con punteros a int, char y Persona. La función mayor() está sobrecargada: compara valores en enteros y caracteres, y compara la edad en Persona.

- Lo que imprime: con los datos de ejemplo, los enteros originales 9 2 7 se recorren ordenados como 2 7 9, los caracteres q d a como a d q, y las personas se muestran por edad: Alicia (22), Carlos (30) y Roberto (45). El resultado es el mismo en PE y POO; solo cambia el encabezado “Resultados PE” / “Resultados POO”.

- Errores y Soluciones: no hubo errores.

Ejercicio 20 - Ordenamiento de manera indirecta (Merge Sort)

Objetivo del código

Implementar el algoritmo de ordenamiento por mezcla (Merge Sort) de manera indirecta para caracteres, enteros y un nuevo tipo de dato abstracto (Persona), aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). En lugar de mover los datos, se ordena un arreglo de punteros que apuntan a ellos, por lo que el arreglo original permanece sin cambios. El programa permite la captura dinámica de los datos por consola antes de ordenarlos.

Código PE

Ejercicio20_PE.cpp | Ejercicio20_POO.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

struct Persona {
    string nombre;
    int edad;
};

bool mayor(int a, int b)         { return a > b; }
bool mayor(char a, char b)       { return a > b; }
bool mayor(Persona a, Persona b) { return a.edad > b.edad; }

template <typename T>
    void mezclar(T* ptr[], int ini, int med, int fin) {
    T* aux[MAX];
    int i = ini, j = med + 1, k = 0;
    while (i <= med && j <= fin) {
        if (mayor(*ptr[i], *ptr[j])) aux[k++] = ptr[j++];
        else                         aux[k++] = ptr[i++];
    }
    while (i <= med) aux[k++] = ptr[i++];
    while (j <= fin) aux[k++] = ptr[j++];
    for (k = 0; k <= fin - ini; k++) ptr[ini + k] = aux[k];
}

    template <typename T>
        void mergeSortIndirecto(T* ptr[], int ini, int fin) {
        if (ini >= fin) return;
        int med = (ini + fin) / 2;
        mergeSortIndirecto(ptr, ini, med);
        mergeSortIndirecto(ptr, med + 1, fin);
        mezclar(ptr, ini, med, fin);
    }

        int main() {
            int n;
            int enteros[MAX];
            char caracteres[MAX];
            Persona personas[MAX];
            int* ptrEnteros[MAX];
            char* ptrCaracteres[MAX];
            Persona* ptrPersonas[MAX];

            cout << "Cantidad de elementos (max 100): ";
            cin >> n;

            cout << "\n--- Captura de Enteros ---\n";
            for (int i = 0; i < n; i++) {
                cout << "Ingresa entero " << i + 1 << ": ";
                cin >> enteros[i];
                ptrEnteros[i] = &enteros[i];
```

```
        }
        cout << "\n--- Captura de Caracteres ---\n";
        for (int i = 0; i < n; i++) {
            cout << "Ingresa caracter " << i + 1 << ": ";
            cin >> caracteres[i];
            ptrCaracteres[i] = &caracteres[i];
        }
        cout << "\n--- Captura de Personas ---\n";
        for (int i = 0; i < n; i++) {
            cout << "Nombre de persona " << i + 1 << ": ";
            cin >> personas[i].nombre;
            cout << "Edad de persona " << i + 1 << ": ";
            cin >> personas[i].edad;
            ptrPersonas[i] = &personas[i];
        }

        mergeSortIndirecto(ptrEnteros, 0, n - 1);
        mergeSortIndirecto(ptrCaracteres, 0, n - 1);
        mergeSortIndirecto(ptrPersonas, 0, n - 1);

        cout << "\nResultados PE\n";
        cout << "Enteros originales: ";
        for (int i = 0; i < n; i++) cout << enteros[i] << " ";
        cout << "\nEnteros ordenados (indirecto): ";
        for (int i = 0; i < n; i++) cout << *ptrEnteros[i] << " ";

        cout << "\nCaracteres originales: ";
        for (int i = 0; i < n; i++) cout << caracteres[i] << " ";
        cout << "\nCaracteres ordenados (indirecto): ";
        for (int i = 0; i < n; i++) cout << *ptrCaracteres[i] << " ";

        cout << "\nPersonas originales: ";
        for (int i = 0; i < n; i++) cout << personas[i].nombre << " (" << personas[i].edad << ") ";
        cout << "\nPersonas ordenadas por edad (indirecto): ";
        for (int i = 0; i < n; i++) cout << ptrPersonas[i]->nombre << " (" << ptrPersonas[i]->edad << ") ";
        cout << endl;
        return 0;
    }

```

Salida

```
D:\Programas\Nueva carpeta\
Cantidad de elementos (max 100): 4

--- Captura de Enteros ---
Ingresa entero 1: 6
Ingresa entero 2: 1
Ingresa entero 3: 8
Ingresa entero 4: 3

--- Captura de Caracteres ---
Ingresa caracter 1: m
Ingresa caracter 2: b
Ingresa caracter 3: x
Ingresa caracter 4: e

--- Captura de Personas ---
Nombre de persona 1: Roberto
Edad de persona 1: 45
Nombre de persona 2: Alicia
Edad de persona 2: 22
Nombre de persona 3: Carlos
Edad de persona 3: 30
Nombre de persona 4: Diana
Edad de persona 4: 19

Resultados PE
Enteros originales: 6 1 8 3
Enteros ordenados (indirecto): 1 3 6 8
Caracteres originales: m b x e
Caracteres ordenados (indirecto): b e m x
Personas originales: Roberto (45) Alicia (22) Carlos (30) Diana (19)
Personas ordenadas por edad (indirecto): Diana (19) Alicia (22) Carlos (30) Roberto (45)


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Código POO

Ejercicio20_POO.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

class Persona {
private:
    string nombre;
    int edad;
public:
    void setNombre(string n) { nombre = n; }
    void setEdad(int e)      { edad = e; }
    string getNombre()       { return nombre; }
    int getEdad()            { return edad; }
};

class OrdenadorMergeIndirecto {
private:
    bool mayor(int a, int b)         { return a > b; }
    bool mayor(char a, char b)       { return a > b; }
    bool mayor(Persona a, Persona b) { return a.getEdad() > b.getEdad(); }

    template <typename T>
    void mezclar(T* ptr[], int ini, int med, int fin) {
        T* aux[MAX];
        int i = ini, j = med + 1, k = 0;
        while (i <= med && j <= fin) {
            if (mayor(*ptr[i], *ptr[j])) aux[k++] = ptr[j++];
            else                         aux[k++] = ptr[i++];
        }
        while (i <= med) aux[k++] = ptr[i++];
        while (j <= fin) aux[k++] = ptr[j++];
        for (k = 0; k <= fin - ini; k++) ptr[ini + k] = aux[k];
    }

    template <typename T>
    void mergeSort(T* ptr[], int ini, int fin) {
        if (ini >= fin) return;
        int med = (ini + fin) / 2;
        mergeSort(ptr, ini, med);
        mergeSort(ptr, med + 1, fin);
        mezclar(ptr, ini, med, fin);
    }
public:
        template <typename T>
        void ordenar(T* ptr[], int n) { mergeSort(ptr, 0, n - 1); }
};
```

```cpp
int main() {
    int n;
    int enteros[MAX];
    char caracteres[MAX];
    Persona personas[MAX];
    int* ptrEnteros[MAX];
    char* ptrCaracteres[MAX];
    Persona* ptrPersonas[MAX];
    OrdenadorMergeIndirecto ordenador;

    cout << "Cantidad de elementos (max 100): ";
    cin >> n;

    cout << "\n--- Captura de Enteros ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa entero " << i + 1 << ": ";
        cin >> enteros[i];
        ptrEnteros[i] = &enteros[i];
    }
    cout << "\n--- Captura de Caracteres ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa caracter " << i + 1 << ": ";
        cin >> caracteres[i];
        ptrCaracteres[i] = &caracteres[i];
    }
    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < n; i++) {
        string nom;
        int e;
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> nom;
        cout << "Edad de persona " << i + 1 << ": ";
        cin >> e;
        personas[i].setNombre(nom);
        personas[i].setEdad(e);
        ptrPersonas[i] = &personas[i];
    }

    ordenador.ordenar(ptrEnteros, n);
    ordenador.ordenar(ptrCaracteres, n);
    ordenador.ordenar(ptrPersonas, n);

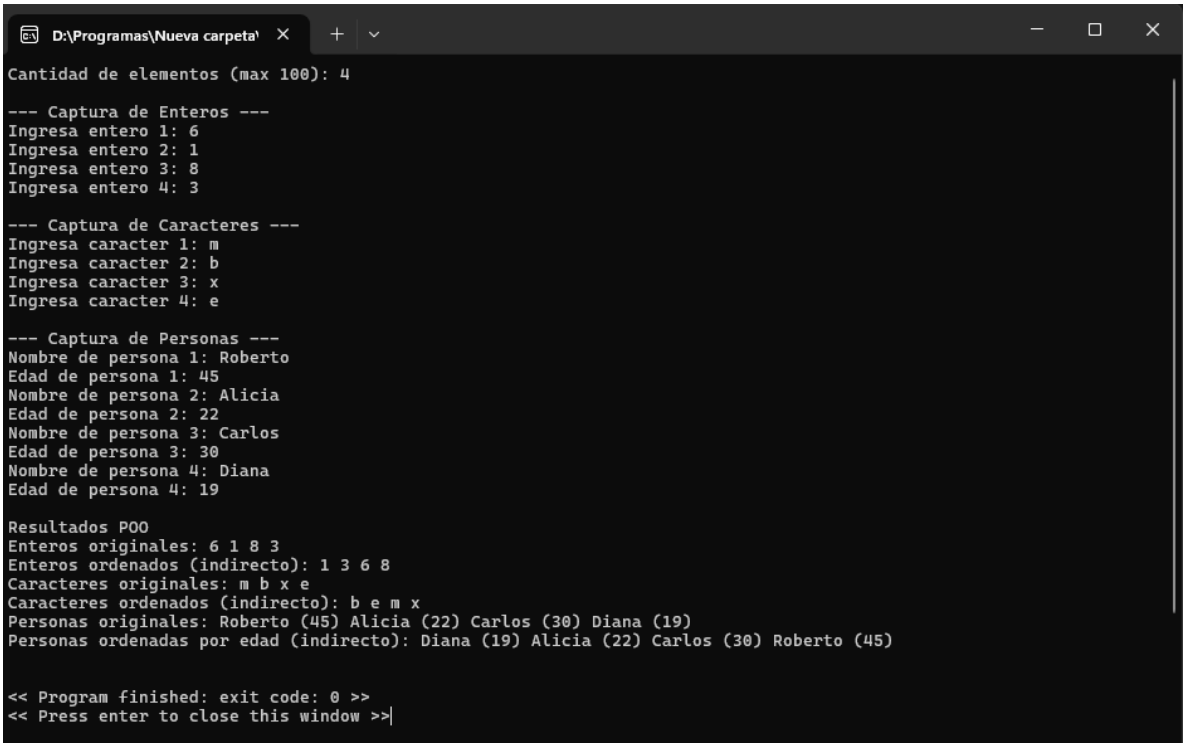
    cout << "\nResultados POO\n";
    cout << "Enteros originales: ";
    for (int i = 0; i < n; i++) cout << enteros[i] << " ";
    cout << "\nEnteros ordenados (indirecto): ";
    for (int i = 0; i < n; i++) cout << *ptrEnteros[i] << " ";

    cout << "\nCaracteres originales: ";
    for (int i = 0; i < n; i++) cout << caracteres[i] << " ";
```

```cpp
    cout << "\nCaracteres ordenados (indirecto): ";
    for (int i = 0; i < n; i++) cout << *ptrCaracteres[i] << " ";

    cout << "\nPersonas originales: ";
    for (int i = 0; i < n; i++)
        cout << personas[i].getNombre() << " (" << personas[i].getEdad() << ") ";
    cout << "\nPersonas ordenadas por edad (indirecto): ";
    for (int i = 0; i < n; i++)
        cout << ptrPersonas[i]->getNombre() << " (" << ptrPersonas[i]->getEdad() << ") ";
    cout << endl;
    return 0;
}
```

Salida



Explicación del Código

- Ingreso de datos dinámico: primero se pide la cantidad de elementos (n) y después se capturan con ciclos for y cin >> los enteros, los caracteres y las personas. Al capturar cada dato se guarda su dirección de memoria (&) en el arreglo de punteros correspondiente.

- Ordenamiento indirecto: Merge Sort no mueve los datos; solo reacomoda los punteros del arreglo auxiliar. Para comparar se accede al valor apuntado con el operador de desreferencia (*ptr[i]), y al terminar los punteros quedan apuntando a los datos en orden.

- Divide y vencerás: mergeSort() divide el arreglo de punteros por la mitad de forma recursiva hasta que cada parte tiene un solo elemento (caso base: ini >= fin).

- Mezcla: mezclar() combina dos mitades ya ordenadas. Compara los datos apuntados por el primer puntero de cada mitad, copia el puntero del menor a un arreglo auxiliar de punteros y al final copia el resultado de vuelta al arreglo de punteros original.

- Arreglo original intacto: como solo se reacomodan los punteros, los arreglos enteros, caracteres y personas conservan su orden de captura. Por eso el programa imprime primero el arreglo original y después los datos recorridos a través de los punteros ya ordenados.

- Acceso a los campos de Persona: en el arreglo de punteros se usa el operador flecha (ptrPersonas[i]->nombre en PE, ptrPersonas[i]->getNombre() en POO) para leer los datos de la persona a la que apunta cada puntero.

- Enfoque PE: los datos de la persona se guardan directamente en los campos de la estructura (personas[i].nombre y personas[i].edad). Las funciones mergeSortIndirecto() y mezclar() son globales.

- Enfoque POO: los datos de la persona se guardan primero en variables temporales (nom y e) y después se asignan con los métodos setNombre() y setEdad(), de modo que los atributos permanecen privados (encapsulamiento). La clase OrdenadorMergeIndirecto contiene el algoritmo y los criterios de comparación como métodos privados, y expone un único método público ordenar().

- Un solo algoritmo para tres tipos de dato: template <typename T> permite usar el mismo código con punteros a int, char y Persona. La función mayor() está sobrecargada: compara valores en enteros y caracteres, y compara la edad en Persona.

- Lo que imprime: con los datos de ejemplo, los enteros originales 6 1 8 3 se recorren ordenados como 1 3 6 8, los caracteres m b x e como b e m x, y las personas se muestran por edad: Diana (19), Alicia (22), Carlos (30) y Roberto (45). El resultado es el mismo en PE y POO; solo cambia el encabezado “Resultados PE” / “Resultados POO”.

- Errores y Soluciones: no hubo errores.

Ejercicio 21 - Ordenamiento de manera indirecta (Quick Sort) con Ingreso de Datos

Objetivo del código

Implementar el algoritmo de ordenamiento rápido (Quick Sort) de manera indirecta para caracteres, enteros y un nuevo tipo de dato abstracto (Persona), aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). En lugar de mover los datos, se ordena un arreglo de punteros que apuntan a ellos, por lo que el arreglo original permanece sin cambios. El programa permite el ingreso de los datos por parte del usuario mediante la consola antes de ordenarlos.

Código PE

Ejercicio21_PE.cpp | Ejercicio21_POO.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

struct Persona {
    string nombre;
    int edad;
};

bool mayor(int a, int b)         { return a > b; }
bool mayor(char a, char b)       { return a > b; }
bool mayor(Persona a, Persona b) { return a.edad > b.edad; }

template <typename T>
    int particion(T* ptr[], int ini, int fin) {
    T* pivote = ptr[fin];
    int i = ini - 1;
    for (int j = ini; j < fin; j++) {
        if (mayor(*pivote, *ptr[j])) {
            i++;
            T* aux = ptr[i];
            ptr[i] = ptr[j];
            ptr[j] = aux;
        }
    }
    T* aux = ptr[i + 1];
    ptr[i + 1] = ptr[fin];
    ptr[fin] = aux;
    return i + 1;
}

    template <typename T>
        void quickSortIndirecto(T* ptr[], int ini, int fin) {
        if (ini >= fin) return;
        int p = particion(ptr, ini, fin);
        quickSortIndirecto(ptr, ini, p - 1);
        quickSortIndirecto(ptr, p + 1, fin);
    }

        int main() {
            int n;
            int enteros[MAX];
            char caracteres[MAX];
            Persona personas[MAX];
            int* ptrEnteros[MAX];
            char* ptrCaracteres[MAX];
            Persona* ptrPersonas[MAX];
```

```
    cout << "Cantidad de elementos (max 100): ";
    cin >> n;

    cout << "\n--- Captura de Enteros ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa entero " << i + 1 << ": ";
        cin >> enteros[i];
        ptrEnteros[i] = &enteros[i];
    }
    cout << "\n--- Captura de Caracteres ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa caracter " << i + 1 << ": ";
        cin >> caracteres[i];
        ptrCaracteres[i] = &caracteres[i];
    }
    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> personas[i].nombre;
        cout << "Edad de persona " << i + 1 << ": ";
        cin >> personas[i].edad;
        ptrPersonas[i] = &personas[i];
    }

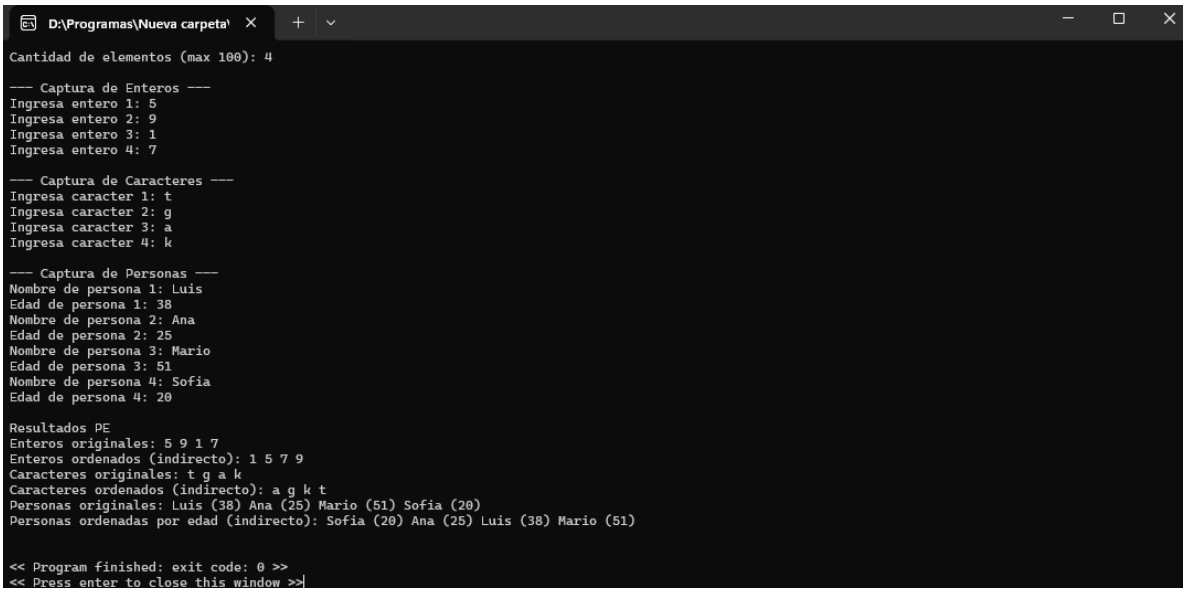
    quickSortIndirecto(ptrEnteros, 0, n - 1);
    quickSortIndirecto(ptrCaracteres, 0, n - 1);
    quickSortIndirecto(ptrPersonas, 0, n - 1);

    cout << "\nResultados PE\n";
    cout << "Enteros originales: ";
    for (int i = 0; i < n; i++) cout << enteros[i] << " ";
    cout << "\nEnteros ordenados (indirecto): ";
    for (int i = 0; i < n; i++) cout << *ptrEnteros[i] << " ";

    cout << "\nCaracteres originales: ";
    for (int i = 0; i < n; i++) cout << caracteres[i] << " ";
    cout << "\nCaracteres ordenados (indirecto): ";
    for (int i = 0; i < n; i++) cout << *ptrCaracteres[i] << " ";

    cout << "\nPersonas originales: ";
    for (int i = 0; i < n; i++) cout << personas[i].nombre << " (" << personas[i].edad << ") ";
    cout << "\nPersonas ordenadas por edad (indirecto): ";
    for (int i = 0; i < n; i++) cout << ptrPersonas[i]->nombre << " (" << ptrPersonas[i]->edad << ") ";
    cout << endl;
    return 0;
}
```

Salida



Código POO

Ejercicio21_POO.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

class Persona {
private:
    string nombre;
    int edad;
public:
    void setNombre(string n) { nombre = n; }
    void setEdad(int e)      { edad = e; }
    string getNombre()       { return nombre; }
    int getEdad()            { return edad; }
};

class OrdenadorQuickIndirecto {
private:
    bool mayor(int a, int b)         { return a > b; }
    bool mayor(char a, char b)       { return a > b; }
    bool mayor(Persona a, Persona b) { return a.getEdad() > b.getEdad(); }

    template <typename T>
    int particion(T* ptr[], int ini, int fin) {
    T* pivote = ptr[fin];
    int i = ini - 1;
    for (int j = ini; j < fin; j++) {
        if (mayor(*pivote, *ptr[j])) {
            i++;
            T* aux = ptr[i];
            ptr[i] = ptr[j];
            ptr[j] = aux;
        }
    }
    T* aux = ptr[i + 1];
    ptr[i + 1] = ptr[fin];
    ptr[fin] = aux;
    return i + 1;
    }

        template <typename T>
        void quickSort(T* ptr[], int ini, int fin) {
        if (ini >= fin) return;
        int p = particion(ptr, ini, fin);
        quickSort(ptr, ini, p - 1);
        quickSort(ptr, p + 1, fin);
        }

public:
            template <typename T>
            void ordenar(T* ptr[], int n) { quickSort(ptr, 0, n - 1); }
```

```
};

int main() {
    int n;
    int enteros[MAX];
    char caracteres[MAX];
    Persona personas[MAX];
    int* ptrEnteros[MAX];
    char* ptrCaracteres[MAX];
    Persona* ptrPersonas[MAX];
    OrdenadorQuickIndirecto ordenador;

    cout << "Cantidad de elementos (max 100): ";
    cin >> n;

    cout << "\n--- Captura de Enteros ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa entero " << i + 1 << ": ";
        cin >> enteros[i];
        ptrEnteros[i] = &enteros[i];
    }
    cout << "\n--- Captura de Caracteres ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Ingresa caracter " << i + 1 << ": ";
        cin >> caracteres[i];
        ptrCaracteres[i] = &caracteres[i];
    }
    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < n; i++) {
        string nom;
        int e;
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> nom;
        cout << "Edad de persona " << i + 1 << ": ";
        cin >> e;
        personas[i].setNombre(nom);
        personas[i].setEdad(e);
        ptrPersonas[i] = &personas[i];
    }

    ordenador.ordenar(ptrEnteros, n);
    ordenador.ordenar(ptrCaracteres, n);
    ordenador.ordenar(ptrPersonas, n);

    cout << "\nResultados POO\n";
    cout << "Enteros originales: ";
    for (int i = 0; i < n; i++) cout << enteros[i] << " ";
    cout << "\nEnteros ordenados (indirecto): ";
```

```
100         cout << "\nEnteros ordenados (indirecto): ";
101         for (int i = 0; i < n; i++) cout << *ptrEnteros[i] << " ";
102
103         cout << "\nCaracteres originales: ";
104         for (int i = 0; i < n; i++) cout << caracteres[i] << " ";
105         cout << "\nCaracteres ordenados (indirecto): ";
106         for (int i = 0; i < n; i++) cout << *ptrCaracteres[i] << " ";
107
108         cout << "\nPersonas originales: ";
109         for (int i = 0; i < n; i++)
110             cout << personas[i].getNombre() << " (" << personas[i].getEdad() << ") ";
111         cout << "\nPersonas ordenadas por edad (indirecto): ";
112         for (int i = 0; i < n; i++)
113             cout << ptrPersonas[i]->getNombre() << " (" << ptrPersonas[i]->getEdad() << ") ";
114         cout << endl;
115         return 0;
116     }
117
```

Salida

```
D:\Programas\Nueva carpeta\
Cantidad de elementos (max 100): 4

--- Captura de Enteros ---
Ingresa entero 1: 5
Ingresa entero 2: 9
Ingresa entero 3: 1
Ingresa entero 4: 7

--- Captura de Caracteres ---
Ingresa caracter 1: t
Ingresa caracter 2: g
Ingresa caracter 3: a
Ingresa caracter 4: k

--- Captura de Personas ---
Nombre de persona 1: Luis
Edad de persona 1: 38
Nombre de persona 2: Ana
Edad de persona 2: 25
Nombre de persona 3: Mario
Edad de persona 3: 51
Nombre de persona 4: Sofia
Edad de persona 4: 20

Resultados POO
Enteros originales: 5 9 1 7
Enteros ordenados (indirecto): 1 5 7 9
Caracteres originales: t g a k
Caracteres ordenados (indirecto): a g k t
Personas originales: Luis (38) Ana (25) Mario (51) Sofia (20)
Personas ordenadas por edad (indirecto): Sofia (20) Ana (25) Luis (38) Mario (51)


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Explicación del Código

- Ingreso de datos dinámico: primero se pide la cantidad de elementos (n) y después se capturan con ciclos for y cin >> los enteros, los caracteres y las personas. Al capturar cada dato se guarda su dirección de memoria (&) en el arreglo de punteros correspondiente.

- Ordenamiento indirecto: Quick Sort no mueve los datos; solo intercambia los punteros del arreglo auxiliar. Para comparar se accede al valor apuntado con el operador de desreferencia (*pivote y *ptr[j]), y al terminar los punteros quedan apuntando a los datos en orden.

- Divide y vencerás: quickSort() elige como pivote el dato apuntado por el último puntero, lo coloca en su posición final con particion() y se llama de forma recursiva sobre la parte izquierda (menores) y la parte derecha (mayores). El caso base es ini >= fin.

- Partición: particion() recorre el arreglo de punteros y mueve al inicio los punteros cuyos datos son menores que el pivote mediante intercambios con una variable auxiliar. Al final, el puntero del pivote se intercambia a la posición que le corresponde y esa posición se devuelve.

- Arreglo original intacto: como solo se reacomodan los punteros, los arreglos enteros, caracteres y personas conservan su orden de captura. Por eso el programa imprime primero el arreglo original y después los datos recorridos a través de los punteros ya ordenados.

- Acceso a los campos de Persona: en el arreglo de punteros se usa el operador flecha (ptrPersonas[i]->nombre en PE, ptrPersonas[i]->getNombre() en POO) para leer los datos de la persona a la que apunta cada puntero.

- Enfoque PE: los datos de la persona se guardan directamente en los campos de la estructura (personas[i].nombre y personas[i].edad). Las funciones quickSortIndirecto() y particion() son globales.

- Enfoque POO: los datos de la persona se guardan primero en variables temporales (nom y e) y después se asignan con los métodos setNombre() y setEdad(), de modo que los atributos permanecen privados (encapsulamiento). La clase OrdenadorQuickIndirecto contiene el algoritmo y los criterios de comparación como métodos privados, y expone un único método público ordenar().

- Un solo algoritmo para tres tipos de dato: template <typename T> permite usar el mismo código con punteros a int, char y Persona. La función mayor() está sobrecargada: compara valores en enteros y caracteres, y compara la edad en Persona.

- Lo que imprime: con los datos de ejemplo, los enteros originales 5 9 1 7 se recorren ordenados como 1 5 7 9, los caracteres t g a k como a g k t, y las personas se muestran por edad: Sofia (20), Ana (25), Luis (38) y Mario (51). El resultado es el mismo en PE y POO; solo cambia el encabezado “Resultados PE” / “Resultados POO”.

- Errores y Soluciones: no hubo errores.

Ejercicio 1 - Punteros para el manejo indirecto con clases y estructuras

Objetivo del código

Implementar, partiendo de la lista de personas y autos del ejercicio 12, estructuras y clases (Persona y Auto) que contengan un puntero al mismo tipo de dato, aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). Cada elemento apunta al siguiente, de modo que la lista se recorre de manera indirecta a través de los punteros, sin usar el índice del arreglo. El programa permite el ingreso de los datos por consola.

Código PE

Ejercicio1_PE.cpp | Ejercicio1_POO.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

struct Persona {
    string nombre, ap, am;
    char genero;
    int edad;
    Persona* siguiente;
};

struct Auto {
    float precio;
    int anio;
    Auto* siguiente;
};

void mostrar(Persona* p) {
    cout << p->nombre << " " << p->ap << " " << p->am
        << " | Genero: " << p->genero << " | Edad: " << p->edad << endl;
}

void mostrar(Auto* a) {
    cout << "Precio: $" << a->precio << " | Anio: " << a->anio << endl;
}

int main() {
    int n;
    Persona personas[MAX];
    Auto autos[MAX];

    cout << "Cantidad de personas y autos (max 100): ";
    cin >> n;

    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < n; i++) {
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> personas[i].nombre;
        cout << "Apellido paterno: ";
        cin >> personas[i].ap;
        cout << "Apellido materno: ";
        cin >> personas[i].am;
        cout << "Genero (M/F): ";
        cin >> personas[i].genero;
        cout << "Edad: ";
        cin >> personas[i].edad;
        personas[i].siguiente = nullptr;
        if (i > 0) personas[i - 1].siguiente = &personas[i];
    }
```

```
52
53      cout << "\n--- Captura de Autos ---\n";
54      for (int i = 0; i < n; i++) {
55          cout << "Precio del auto " << i + 1 << ": ";
56          cin >> autos[i].precio;
57          cout << "Anio del auto " << i + 1 << ": ";
58          cin >> autos[i].anio;
59          autos[i].siguiente = nullptr;
60          if (i > 0) autos[i - 1].siguiente = &autos[i];
61      }
62
63      cout << "\nResultados PE\n";
64      cout << "--- Personas (recorridas con punteros) ---\n";
65      Persona* p = n > 0 ? personas : nullptr;
66      while (p != nullptr) {
67          mostrar(p);
68          p = p->siguiente;
69      }
70
71      cout << "--- Autos (recorridos con punteros) ---\n";
72      Auto* a = n > 0 ? autos : nullptr;
73      while (a != nullptr) {
74          mostrar(a);
75          a = a->siguiente;
76      }
77      return 0;
78  }
79
```

Salida

```
D:\Programas\Nueva carpeta\

Cantidad de personas y autos (max 100): 3

--- Captura de Personas ---
Nombre de persona 1: Luis
Apellido paterno: Perez
Apellido materno: Gomez
Genero (M/F): M
Edad: 38
Nombre de persona 2: Ana
Apellido paterno: Lopez
Apellido materno: Ruiz
Genero (M/F): F
Edad: 25
Nombre de persona 3: Mario
Apellido paterno: Diaz
Apellido materno: Soto
Genero (M/F): M
Edad: 51

--- Captura de Autos ---
Precio del auto 1: 250000
Anio del auto 1: 2018
Precio del auto 2: 180000
Anio del auto 2: 2015
Precio del auto 3: 320000
Anio del auto 3: 2022

Resultados PE
--- Personas (recorridas con punteros) ---
Luis Perez Gomez | Genero: M | Edad: 38
Ana Lopez Ruiz | Genero: F | Edad: 25
Mario Diaz Soto | Genero: M | Edad: 51
--- Autos (recorridos con punteros) ---
Precio: $250000 | Anio: 2018
Precio: $180000 | Anio: 2015
Precio: $320000 | Anio: 2022


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Código POO

Ejercicio1_POO.cpp

```cpp
#include <iostream>
#include <string>
using namespace std;

const int MAX = 100;

class Persona {
private:
    string nombre, ap, am;
    char genero;
    int edad;
    Persona* siguiente;
public:
    Persona() { siguiente = nullptr; }
    void setDatos(string n, string a, string m, char g, int e) {
        nombre = n;
        ap = a;
        am = m;
        genero = g;
        edad = e;
    }
    void setSiguiente(Persona* s) { siguiente = s; }
    Persona* getSiguiente()       { return siguiente; }
    void mostrar() {
        cout << nombre << " " << ap << " " << am
             << " | Genero: " << genero << " | Edad: " << edad << endl;
    }
};

class Auto {
private:
    float precio;
    int anio;
    Auto* siguiente;
public:
    Auto() { siguiente = nullptr; }
    void setDatos(float p, int a) {
        precio = p;
        anio = a;
    }
    void setSiguiente(Auto* s) { siguiente = s; }
    Auto* getSiguiente()       { return siguiente; }
    void mostrar() {
        cout << "Precio: $" << precio << " | Anio: " << anio << endl;
    }
};

int main() {
    int n;
    Persona personas[MAX];
    Auto autos[MAX];

```

```
    cout << "Cantidad de personas y autos (max 100): ";
    cin >> n;

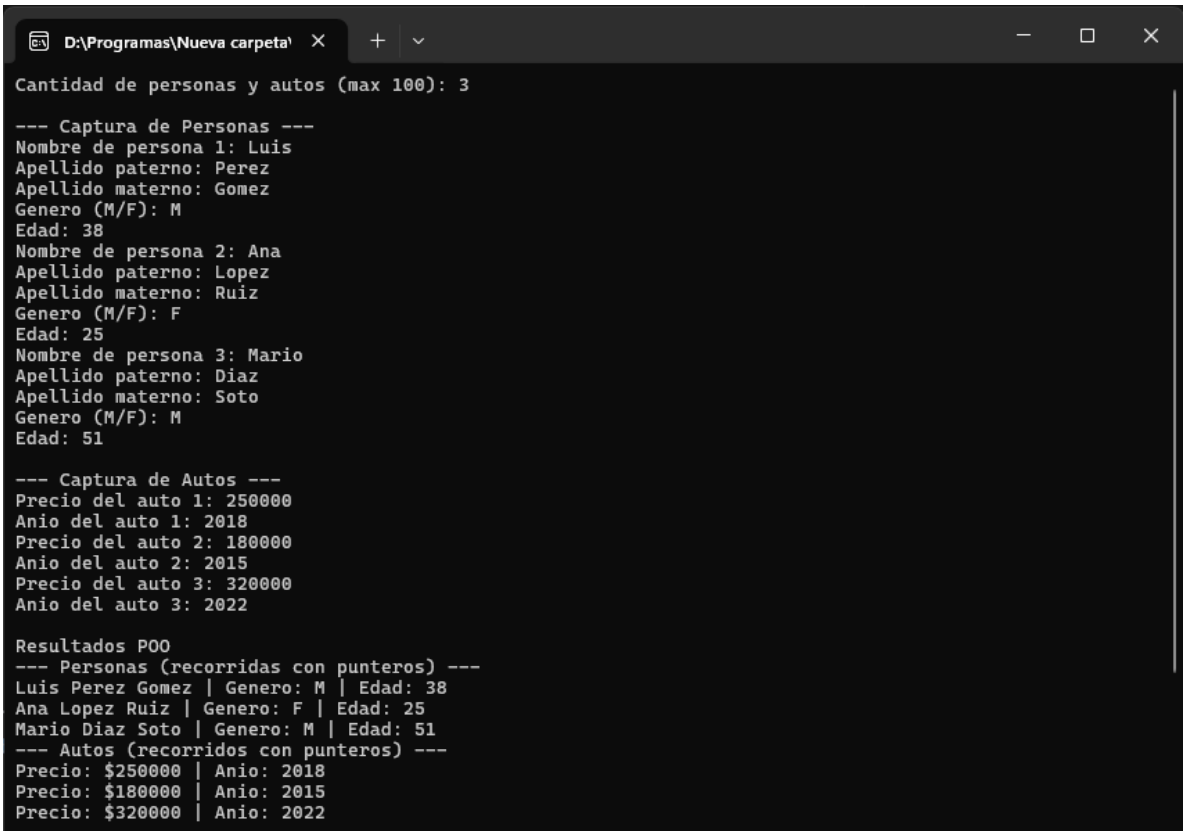
    cout << "\n--- Captura de Personas ---\n";
    for (int i = 0; i < n; i++) {
        string nom, ap, am;
        char gen;
        int edad;
        cout << "Nombre de persona " << i + 1 << ": ";
        cin >> nom;
        cout << "Apellido paterno: ";
        cin >> ap;
        cout << "Apellido materno: ";
        cin >> am;
        cout << "Genero (M/F): ";
        cin >> gen;
        cout << "Edad: ";
        cin >> edad;
        personas[i].setDatos(nom, ap, am, gen, edad);
        if (i > 0) personas[i - 1].setSiguiente(&personas[i]);
    }

    cout << "\n--- Captura de Autos ---\n";
    for (int i = 0; i < n; i++) {
        float precio;
        int anio;
        cout << "Precio del auto " << i + 1 << ": ";
        cin >> precio;
        cout << "Anio del auto " << i + 1 << ": ";
        cin >> anio;
        autos[i].setDatos(precio, anio);
        if (i > 0) autos[i - 1].setSiguiente(&autos[i]);
    }

    cout << "\nResultados POO\n";
    cout << "--- Personas (recorridas con punteros) ---\n";
    Persona* p = n > 0 ? personas : nullptr;
    while (p != nullptr) {
        p->mostrar();
        p = p->getSiguiente();
    }

    cout << "--- Autos (recorridos con punteros) ---\n";
    Auto* a = n > 0 ? autos : nullptr;
    while (a != nullptr) {
        a->mostrar();
        a = a->getSiguiente();
    }
    return 0;
}
```

Salida



Explicación del Código

- Ingreso de datos dinámico: primero se pide la cantidad de elementos (n) y después se capturan con ciclos for y cin >> los datos de las personas (nombre, apellidos, género y edad) y de los autos (precio y año).
- Puntero al mismo tipo de dato: cada Persona contiene un puntero Persona* siguiente y cada Auto contiene un puntero Auto* siguiente. Es una estructura autorreferenciada, es decir, un elemento puede guardar la dirección de otro elemento de su mismo tipo.
- Enlace de los elementos: al capturar el elemento i, el elemento anterior (i - 1) guarda su dirección con el operador & en su puntero siguiente. El último elemento conserva nullptr, lo que indica el final de la lista.
- Recorrido indirecto: se crea un puntero auxiliar (p o a) que inicia en el primer elemento y avanza con p = p->siguiente hasta llegar a nullptr. Así los datos se leen a través de los punteros y no con el índice del arreglo.
- Operador flecha: p->nombre y p->siguiente acceden a los campos del elemento al que apunta el puntero. Es equivalente a (*p).nombre.
- Enfoque PE: Persona y Auto son estructuras con todos sus campos públicos, y las funciones mostrar() son globales y se reciben el puntero como parámetro (sobrecarga de funciones).
- Enfoque POO: los atributos son privados y se asignan con setDatos() y setSiguiente() (encapsulamiento). El constructor inicializa siguiente en nullptr, y cada clase tiene su propio método mostrar() para imprimirse.

- Lo que imprime: con los datos de ejemplo, el programa recorre la lista de personas desde Luis hasta Mario y la lista de autos desde el de 2018 hasta el de 2022, siguiendo los punteros en el mismo orden de captura. El resultado es el mismo en PE y POO; solo cambia el encabezado “Resultados PE” / “Resultados POO”.

- Errores y Soluciones: no hubo errores.

Ejercicio 3 - Pila estática por medio de un arreglo (PE y POO)

Objetivo del código

Implementar una pila (estructura LIFO, último en entrar, primero en salir) de manera estática por medio de un arreglo de tamaño fijo, aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). El programa permite al usuario, mediante un menú en consola, apilar (push), desapilar (pop), consultar la cima y mostrar el contenido de la pila, controlando los casos de pila llena y pila vacía.

Código PE

Ejercicio3_PE.cpp | Ejercicio3_POO.cpp

```cpp
#include <iostream>
using namespace std;

const int MAX = 5;

struct Pila {
    int datos[MAX];
    int tope;
};

void iniciar(Pila &p) { p.tope = -1; }
bool vacia(Pila &p)   { return p.tope == -1; }
bool llena(Pila &p)   { return p.tope == MAX - 1; }

bool push(Pila &p, int dato) {
    if (llena(p)) return false;
    p.datos[++p.tope] = dato;
    return true;
}

bool pop(Pila &p, int &dato) {
    if (vacia(p)) return false;
    dato = p.datos[p.tope--];
    return true;
}

bool cima(Pila &p, int &dato) {
    if (vacia(p)) return false;
    dato = p.datos[p.tope];
    return true;
}

void mostrar(Pila &p) {
    if (vacia(p)) {
        cout << "La pila esta vacia\n";
        return;
    }
    cout << "Pila (de la cima a la base): ";
    for (int i = p.tope; i >= 0; i--) cout << p.datos[i] << " ";
    cout << endl;
}

```

```cpp
int main() {
    Pila pila;
    iniciar(pila);
    int opcion, dato;

    cout << "--- Pila Estatica PE (capacidad " << MAX << ") ---\n";
    do {
        cout << "\n1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Dato a apilar: ";
            cin >> dato;
            if (push(pila, dato)) cout << "Dato apilado\n";
            else cout << "Pila llena: no se puede apilar\n";
            break;
        case 2:
            if (pop(pila, dato)) cout << "Dato desapilado: " << dato << endl;
            else cout << "Pila vacia: no se puede desapilar\n";
            break;
        case 3:
            if (cima(pila, dato)) cout << "Cima: " << dato << endl;
            else cout << "Pila vacia\n";
            break;
        case 4:
            mostrar(pila);
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}
```

Salida

```
D:\Programas\Nueva ca

--- Pila Estatica PE (capacidad 5) ---

1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 1
Dato a apilar: 10
Dato apilado

1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 1
Dato a apilar: 20
Dato apilado

1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 1
Dato a apilar: 30
Dato apilado

1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 4
Pila (de la cima a la base): 30 20 10

1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 3
Cima: 30

1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 2
Dato desapilado: 30

1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 4
Pila (de la cima a la base): 20 10

1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 0
Fin del programa


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Código POO

Ejercicio3_POO.cpp

```cpp
#include <iostream>
using namespace std;

const int MAX = 5;

class PilaEstatica {
private:
    int datos[MAX];
    int tope;
public:
    PilaEstatica() { tope = -1; }

    bool vacia() { return tope == -1; }
    bool llena() { return tope == MAX - 1; }

    bool push(int dato) {
        if (llena()) return false;
        datos[++tope] = dato;
        return true;
    }

    bool pop(int &dato) {
        if (vacia()) return false;
        dato = datos[tope--];
        return true;
    }

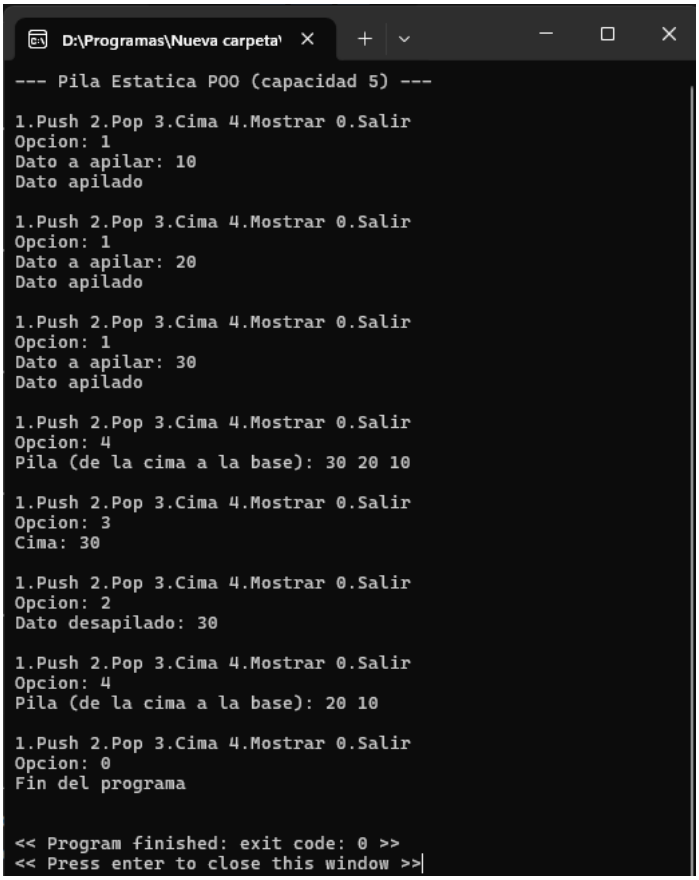
    bool cima(int &dato) {
        if (vacia()) return false;
        dato = datos[tope];
        return true;
    }

    void mostrar() {
        if (vacia()) {
            cout << "La pila esta vacia\n";
            return;
        }
        cout << "Pila (de la cima a la base): ";
        for (int i = tope; i >= 0; i--) cout << datos[i] << " ";
        cout << endl;
    }
};
```

```cpp
int main() {
    PilaEstatica pila;
    int opcion, dato;

    cout << "--- Pila Estatica POO (capacidad " << MAX << ") ---\n";
    do {
        cout << "\n1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Dato a apilar: ";
            cin >> dato;
            if (pila.push(dato)) cout << "Dato apilado\n";
            else cout << "Pila llena: no se puede apilar\n";
            break;
        case 2:
            if (pila.pop(dato)) cout << "Dato desapilado: " << dato << endl;
            else cout << "Pila vacia: no se puede desapilar\n";
            break;
        case 3:
            if (pila.cima(dato)) cout << "Cima: " << dato << endl;
            else cout << "Pila vacia\n";
            break;
        case 4:
            pila.mostrar();
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}
```

Salida



Explicación del Código

- Pila estática: la pila se implementa con un arreglo de tamaño fijo (MAX = 5) y una variable entera tope que guarda la posición del último dato apilado. Cuando la pila está vacía, tope vale -1.
- Estructura LIFO: el último dato en entrar es el primero en salir. Por eso, después de apilar 10, 20 y 30, el primer dato que se desapila es el 30.
- Push (apilar): primero verifica que la pila no esté llena (tope == MAX - 1). Si hay espacio, incrementa tope y guarda el dato en esa posición. Si no hay espacio, regresa false y el programa avisa que la pila está llena (desbordamiento).
- Pop (desapilar): primero verifica que la pila no esté vacía (tope == -1). Si tiene datos, entrega el dato de la posición tope y después disminuye tope. Si está vacía, regresa false y el programa avisa que no hay nada que desapilar.
- Cima: devuelve el dato del tope sin eliminarlo, por lo que la pila no cambia.
- Mostrar: recorre el arreglo desde tope hasta 0 para imprimir los datos de la cima a la base.

- Parámetros por referencia: en pop() y cima() el dato se devuelve por medio del parámetro &dato, mientras que el valor de retorno (bool) indica si la operación fue exitosa.

- Menú interactivo: el ciclo do-while repite el menú hasta que el usuario elige 0, y el switch ejecuta la operación elegida.

- Enfoque PE: Pila es una estructura con los campos datos y tope, y las operaciones (iniciar, push, pop, cima, mostrar) son funciones globales que reciben la pila por referencia (&).

- Enfoque POO: la clase PilaEstatica encapsula el arreglo y el tope como atributos privados, el constructor inicializa tope en -1, y las operaciones son métodos públicos que trabajan directamente sobre los datos del objeto.

- Lo que imprime: con la secuencia de ejemplo, se apilan 10, 20 y 30, se muestra la pila (30 20 10), se consulta la cima (30), se desapila el 30 y se muestra la pila final (20 10). El resultado es el mismo en PE y POO; solo cambia el título “Pila Estatica PE” / “Pila Estatica POO”.

- Errores y Soluciones: no hubo errores.

Ejercicio 4 - Cola estática por medio de un arreglo (PE y POO)

Objetivo del código

Implementar una cola (estructura FIFO, primero en entrar, primero en salir) de manera estática por medio de un arreglo de tamaño fijo, aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). El programa permite al usuario, mediante un menú en consola, encolar, desencolar, consultar el primer elemento y mostrar el contenido de la cola, controlando los casos de cola llena y cola vacía.

Código PE

Ejercicio4_PE.cpp

```cpp
#include <iostream>
using namespace std;

const int MAX = 5;

struct Cola {
    int datos[MAX];
    int frente;
    int cantidad;
};

void iniciar(Cola &c) {
    c.frente = 0;
    c.cantidad = 0;
}

bool vacia(Cola &c) { return c.cantidad == 0; }
bool llena(Cola &c) { return c.cantidad == MAX; }

bool encolar(Cola &c, int dato) {
    if (llena(c)) return false;
    c.datos[(c.frente + c.cantidad) % MAX] = dato;
    c.cantidad++;
    return true;
}

bool desencolar(Cola &c, int &dato) {
    if (vacia(c)) return false;
    dato = c.datos[c.frente];
    c.frente = (c.frente + 1) % MAX;
    c.cantidad--;
    return true;
}

bool primero(Cola &c, int &dato) {
    if (vacia(c)) return false;
    dato = c.datos[c.frente];
    return true;
}

void mostrar(Cola &c) {
    if (vacia(c)) {
        cout << "La cola esta vacia\n";
        return;
    }
    cout << "Cola (del frente al final): ";
    for (int i = 0; i < c.cantidad; i++)
        cout << c.datos[(c.frente + i) % MAX] << " ";
    cout << endl;
}

```

```cpp
int main() {
    Cola cola;
    iniciar(cola);
    int opcion, dato;

    cout << "--- Cola Estatica PE (capacidad " << MAX << ") ---\n";
    do {
        cout << "\n1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Dato a encolar: ";
            cin >> dato;
            if (encolar(cola, dato)) cout << "Dato encolado\n";
            else cout << "Cola llena: no se puede encolar\n";
            break;
        case 2:
            if (desencolar(cola, dato)) cout << "Dato desencolado: " << dato << endl;
            else cout << "Cola vacia: no se puede desencolar\n";
            break;
        case 3:
            if (primero(cola, dato)) cout << "Primero: " << dato << endl;
            else cout << "Cola vacia\n";
            break;
        case 4:
            mostrar(cola);
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}
```

Salida

```
D:\Programas\Nueva ca

--- Cola Estatica PE (capacidad 5) ---

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 10
Dato encolado

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 20
Dato encolado

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 30
Dato encolado

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 4
Cola (del frente al final): 10 20 30

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 3
Primero: 10

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 2
Dato desencolado: 10

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 4
Cola (del frente al final): 20 30

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 0
Fin del programa


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Código POO

Ejercicio4_POO.cpp

```cpp
#include <iostream>
using namespace std;

const int MAX = 5;

class ColaEstatica {
private:
    int datos[MAX];
    int frente;
    int cantidad;
public:
    ColaEstatica() {
        frente = 0;
        cantidad = 0;
    }

    bool vacia() { return cantidad == 0; }
    bool llena() { return cantidad == MAX; }

    bool encolar(int dato) {
        if (llena()) return false;
        datos[(frente + cantidad) % MAX] = dato;
        cantidad++;
        return true;
    }

    bool desencolar(int &dato) {
        if (vacia()) return false;
        dato = datos[frente];
        frente = (frente + 1) % MAX;
        cantidad--;
        return true;
    }

    bool primero(int &dato) {
        if (vacia()) return false;
        dato = datos[frente];
        return true;
    }

    void mostrar() {
        if (vacia()) {
            cout << "La cola esta vacia\n";
            return;
        }
        cout << "Cola (del frente al final): ";
        for (int i = 0; i < cantidad; i++)
            cout << datos[(frente + i) % MAX] << " ";
        cout << endl;
    }
};

```

```cpp
int main() {
    ColaEstatica cola;
    int opcion, dato;

    cout << "--- Cola Estatica POO (capacidad " << MAX << ") ---\n";
    do {
        cout << "\n1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Dato a encolar: ";
            cin >> dato;
            if (cola.encolar(dato)) cout << "Dato encolado\n";
            else cout << "Cola llena: no se puede encolar\n";
            break;
        case 2:
            if (cola.desencolar(dato)) cout << "Dato desencolado: " << dato << endl;
            else cout << "Cola vacia: no se puede desencolar\n";
            break;
        case 3:
            if (cola.primero(dato)) cout << "Primero: " << dato << endl;
            else cout << "Cola vacia\n";
            break;
        case 4:
            cola.mostrar();
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}
```

Salida

```
D:\Programas\Nueva ca
--- Cola Estatica POO (capacidad 5) ---

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 10
Dato encolado

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 20
Dato encolado

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 30
Dato encolado

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 4
Cola (del frente al final): 10 20 30

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 3
Primero: 10

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 2
Dato desencolado: 10

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 4
Cola (del frente al final): 20 30

1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 0
Fin del programa


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Explicación del Código

- Cola estática: la cola se implementa con un arreglo de tamaño fijo (MAX = 5) y dos variables enteras: frente, que guarda la posición del primer dato, y cantidad, que guarda cuántos datos hay en la cola.
- Estructura FIFO: el primer dato en entrar es el primero en salir. Por eso, después de encolar 10, 20 y 30, el primer dato que se desencola es el 10.
- Arreglo circular: las posiciones se calculan con el operador módulo (% MAX), de modo que al llegar al final del arreglo se regresa al inicio. Así se reutilizan los espacios que quedan libres al desencolar.
- Encolar: primero verifica que la cola no esté llena (cantidad == MAX). Si hay espacio, guarda el dato en la posición (frente + cantidad) % MAX, que es el final de la cola, e incrementa cantidad. Si no hay espacio, regresa false y el programa avisa que la cola está llena.
- Desencolar: primero verifica que la cola no esté vacía (cantidad == 0). Si tiene datos, entrega el dato de la posición frente, avanza frente una posición con (frente + 1) % MAX y disminuye cantidad. Si está vacía, regresa false y el programa avisa que no hay nada que desencolar.
- Primero: devuelve el dato del frente sin eliminarlo, por lo que la cola no cambia.
- Mostrar: recorre la cola desde la posición frente, tantas veces como indique cantidad, para imprimir los datos del frente al final.
- Parámetros por referencia: en desencolar() y primero() el dato se devuelve por medio del parámetro &dato, mientras que el valor de retorno (bool) indica si la operación fue exitosa.
- Menú interactivo: el ciclo do-while repite el menú hasta que el usuario elige 0, y el switch ejecuta la operación elegida.
- Enfoque PE: Cola es una estructura con los campos datos, frente y cantidad, y las operaciones (iniciar, encolar, desencolar, primero, mostrar) son funciones globales que reciben la cola por referencia (&).
- Enfoque POO: la clase ColaEstatica encapsula el arreglo, el frente y la cantidad como atributos privados, el constructor los inicializa en 0, y las operaciones son métodos públicos que trabajan directamente sobre los datos del objeto.
- Lo que imprime: con la secuencia de ejemplo, se encolan 10, 20 y 30, se muestra la cola (10 20 30), se consulta el primero (10), se desencola el 10 y se muestra la cola final (20 30). El resultado es el mismo en PE y POO; solo cambia el título “Cola Estatica PE” / “Cola Estatica POO”.
- Errores y Soluciones: no hubo errores.

Ejercicio 5 - Lista estática por medio de un arreglo (PE y POO)

Objetivo del código

Implementar una lista de manera estática por medio de un arreglo de tamaño fijo, aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). A diferencia de la pila y la cola, la lista permite insertar y eliminar elementos en cualquier posición. El programa permite al usuario, mediante un menú en consola, insertar, eliminar, buscar y mostrar los datos de la lista, controlando los casos de lista llena, lista vacía y posición inválida.

Código PE

Ejercicio5_PE.cpp

```cpp
#include <iostream>
using namespace std;

const int MAX = 5;

struct Lista {
    int datos[MAX];
    int cantidad;
};

void iniciar(Lista &l) { l.cantidad = 0; }
bool vacia(Lista &l)   { return l.cantidad == 0; }
bool llena(Lista &l)   { return l.cantidad == MAX; }

bool insertar(Lista &l, int pos, int dato) {
    if (llena(l) || pos < 1 || pos > l.cantidad + 1) return false;
    for (int i = l.cantidad; i >= pos; i--) l.datos[i] = l.datos[i - 1];
    l.datos[pos - 1] = dato;
    l.cantidad++;
    return true;
}

bool eliminar(Lista &l, int pos, int &dato) {
    if (pos < 1 || pos > l.cantidad) return false;
    dato = l.datos[pos - 1];
    for (int i = pos - 1; i < l.cantidad - 1; i++) l.datos[i] = l.datos[i + 1];
    l.cantidad--;
    return true;
}

int buscar(Lista &l, int dato) {
    for (int i = 0; i < l.cantidad; i++)
        if (l.datos[i] == dato) return i + 1;
    return 0;
}

void mostrar(Lista &l) {
    if (vacia(l)) {
        cout << "La lista esta vacia\n";
        return;
    }
    cout << "Lista: ";
    for (int i = 0; i < l.cantidad; i++) cout << l.datos[i] << " ";
    cout << endl;
}

```

```cpp
int main() {
    Lista lista;
    iniciar(lista);
    int opcion, dato, pos;

    cout << "--- Lista Estatica PE (capacidad " << MAX << ") ---\n";
    do {
        cout << "\n1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Posicion (1 a " << lista.cantidad + 1 << "): ";
            cin >> pos;
            cout << "Dato a insertar: ";
            cin >> dato;
            if (insertar(lista, pos, dato)) cout << "Dato insertado\n";
            else cout << "Lista llena o posicion invalida\n";
            break;
        case 2:
            cout << "Posicion a eliminar: ";
            cin >> pos;
            if (eliminar(lista, pos, dato)) cout << "Dato eliminado: " << dato << endl;
            else cout << "Lista vacia o posicion invalida\n";
            break;
        case 3:
            cout << "Dato a buscar: ";
            cin >> dato;
            pos = buscar(lista, dato);
            if (pos > 0) cout << "Dato " << dato << " encontrado en la posicion " << pos << endl;
            else cout << "Dato no encontrado\n";
            break;
        case 4:
            mostrar(lista);
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}
```

Salida

```
D:\Programas\Nueva c
--- Lista Estatica PE (capacidad 5) ---

1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 1): 1
Dato a insertar: 10
Dato insertado

1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 2): 2
Dato a insertar: 20
Dato insertado

1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 3): 3
Dato a insertar: 30
Dato insertado

1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 4): 2
Dato a insertar: 15
Dato insertado

1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 4
Lista: 10 15 20 30

1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 3
Dato a buscar: 20
Dato 20 encontrado en la posicion 3

1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 2
Posicion a eliminar: 1
Dato eliminado: 10

1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 4
Lista: 15 20 30

1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 0
Fin del programa


<< Program finished: exit code: 0 >>
<< Press enter to close this window >>
```

Código POO

Ejercicio5_PE.cpp | Ejercicio5_POO.cpp

```cpp
#include <iostream>
using namespace std;

const int MAX = 5;

class ListaEstatica {
private:
    int datos[MAX];
    int cantidad;
public:
    ListaEstatica() { cantidad = 0; }

    int getCantidad() { return cantidad; }
    bool vacia()      { return cantidad == 0; }
    bool llena()      { return cantidad == MAX; }

    bool insertar(int pos, int dato) {
        if (llena() || pos < 1 || pos > cantidad + 1) return false;
        for (int i = cantidad; i >= pos; i--) datos[i] = datos[i - 1];
        datos[pos - 1] = dato;
        cantidad++;
        return true;
    }

    bool eliminar(int pos, int &dato) {
        if (pos < 1 || pos > cantidad) return false;
        dato = datos[pos - 1];
        for (int i = pos - 1; i < cantidad - 1; i++) datos[i] = datos[i + 1];
        cantidad--;
        return true;
    }

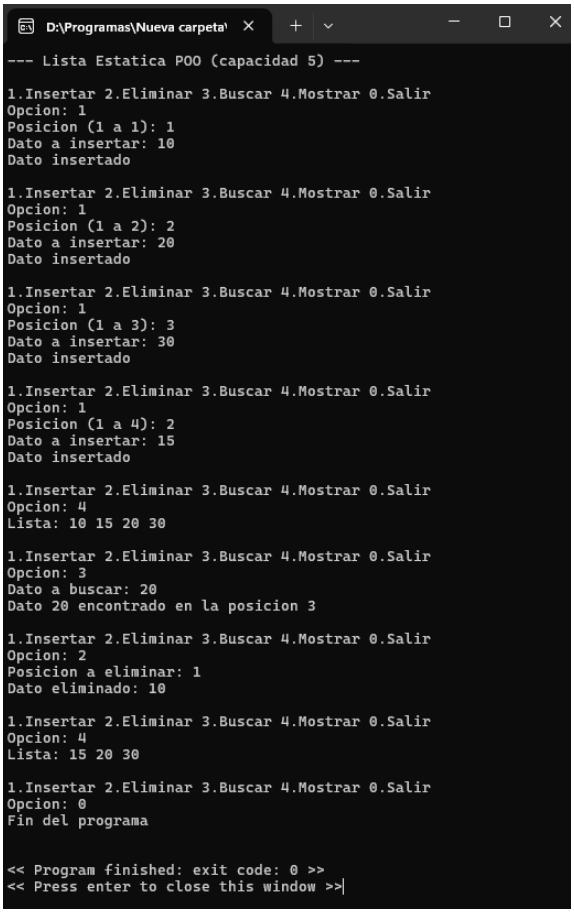
    int buscar(int dato) {
        for (int i = 0; i < cantidad; i++)
            if (datos[i] == dato) return i + 1;
        return 0;
    }

    void mostrar() {
        if (vacia()) {
            cout << "La lista esta vacia\n";
            return;
        }
        cout << "Lista: ";
        for (int i = 0; i < cantidad; i++) cout << datos[i] << " ";
        cout << endl;
    }
};
```

```cpp
int main() {
    ListaEstatica lista;
    int opcion, dato, pos;

    cout << "--- Lista Estatica POO (capacidad " << MAX << ") ---\n";
    do {
        cout << "\n1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Posicion (1 a " << lista.getCantidad() + 1 << "): ";
            cin >> pos;
            cout << "Dato a insertar: ";
            cin >> dato;
            if (lista.insertar(pos, dato)) cout << "Dato insertado\n";
            else cout << "Lista llena o posicion invalida\n";
            break;
        case 2:
            cout << "Posicion a eliminar: ";
            cin >> pos;
            if (lista.eliminar(pos, dato)) cout << "Dato eliminado: " << dato << endl;
            else cout << "Lista vacia o posicion invalida\n";
            break;
        case 3:
            cout << "Dato a buscar: ";
            cin >> dato;
            pos = lista.buscar(dato);
            if (pos > 0) cout << "Dato " << dato << " encontrado en la posicion " << pos << endl;
            else cout << "Dato no encontrado\n";
            break;
        case 4:
            lista.mostrar();
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}
```

Salida



Explicación del Código

- Lista estática: la lista se implementa con un arreglo de tamaño fijo (MAX = 5) y una variable entera cantidad que guarda cuántos datos hay almacenados. Los datos ocupan siempre las primeras posiciones del arreglo, sin huecos.

- Posiciones: el usuario trabaja con posiciones desde 1, y el programa las convierte al índice del arreglo restando 1 (la posición 1 corresponde a datos[0]).

- Insertar: primero verifica que la lista no esté llena y que la posición sea válida (de 1 a cantidad + 1). Después recorre los datos desde el final hacia la posición indicada, moviendo cada uno un lugar a la derecha para hacer espacio guarda el nuevo dato e incrementa cantidad. En el ejemplo, al insertar 15 en la posición 2, los datos 20 y 30 se recorren a la derecha y la lista queda 10 15 20 30.

- Eliminar: primero verifica que la posición sea válida (de 1 a cantidad). Guarda el dato eliminado, recorre los datos posteriores un lugar a la izquierda para tapar el hueco y disminuye cantidad. En el ejemplo, al eliminar la posición 1 se obtiene el 10 y la lista queda 15 20 30.

- Buscar: recorre la lista comparando cada dato con el buscado. Regresa su posición (a partir de 1) si lo encuentra, o 0 si no existe.

- Mostrar: recorre el arreglo desde 0 hasta cantidad - 1 para imprimir todos los datos en orden.

- Diferencia con pila y cola: en la pila solo se inserta y elimina por la cima, y en la cola por los extremos (entra por el final, sale por el frente). En la lista se puede insertar y eliminar en cualquier posición, a costa de recorrer los datos del arreglo.

- Parámetros por referencia: en eliminar() el dato se devuelve por medio del parámetro &dato, mientras que el valor de retorno (bool) indica si la operación fue exitosa.

- Menú interactivo: el ciclo do-while repite el menú hasta que el usuario elige 0, y el switch ejecuta la operación elegida.

- Enfoque PE: Lista es una estructura con los campos datos y cantidad, y las operaciones (iniciar, insertar, eliminar, buscar, mostrar) son funciones globales que reciben la lista por referencia (&).

- Enfoque POO: la clase ListaEstatica encapsula el arreglo y la cantidad como atributos privados, el constructor inicializa cantidad en 0, y las operaciones son métodos públicos que trabajan directamente sobre los datos del objeto.

- Lo que imprime: con la secuencia de ejemplo, se insertan 10, 20 y 30, se inserta 15 en la posición 2, se muestra la lista (10 15 20 30), se busca el 20 (posición 3), se elimina la posición 1 (dato 10) y se muestra la lista final (15 20 30). El resultado es el mismo en PE y POO; solo cambia el título “Lista Estatica PE” / “Lista Estatica POO”.

- Errores y Soluciones: no hubo errores.

Ejercicio 6 - Pila dinámica por medio de punteros al mismo tipo de dato (PE y POO)

Objetivo del código

Implementar una pila (estructura LIFO, último en entrar, primero en salir) de manera dinámica por medio de nodos enlazados con punteros al mismo tipo de dato, aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). A diferencia de la pila estática, no tiene un tamaño máximo fijo: la memoria se reserva con new al apilar y se libera con delete al desapilar. El programa permite al usuario, mediante un menú en consola, apilar (push), desapilar (pop), consultar la cima y mostrar el contenido de la pila, controlando el caso de pila vacía.

Código PE

Ejercicio6_PE.cpp | Ejercicio6_POO.cpp

```cpp
#include <iostream>
using namespace std;

struct Nodo {
    int dato;
    Nodo* siguiente;
};

struct Pila {
    Nodo* tope;
};

void iniciar(Pila &p) { p.tope = nullptr; }
bool vacia(Pila &p)   { return p.tope == nullptr; }

void push(Pila &p, int dato) {
    Nodo* nuevo = new Nodo;
    nuevo->dato = dato;
    nuevo->siguiente = p.tope;
    p.tope = nuevo;
}

bool pop(Pila &p, int &dato) {
    if (vacia(p)) return false;
    Nodo* aux = p.tope;
    dato = aux->dato;
    p.tope = aux->siguiente;
    delete aux;
    return true;
}

bool cima(Pila &p, int &dato) {
    if (vacia(p)) return false;
    dato = p.tope->dato;
    return true;
}

void mostrar(Pila &p) {
    if (vacia(p)) {
        cout << "La pila esta vacia\n";
        return;
    }
    cout << "Pila (de la cima a la base): ";
    Nodo* aux = p.tope;
    while (aux != nullptr) {
        cout << aux->dato << " ";
        aux = aux->siguiente;
    }
    cout << endl;
}
```

```
}

void liberar(Pila &p) {
    int dato;
    while (pop(p, dato));
}

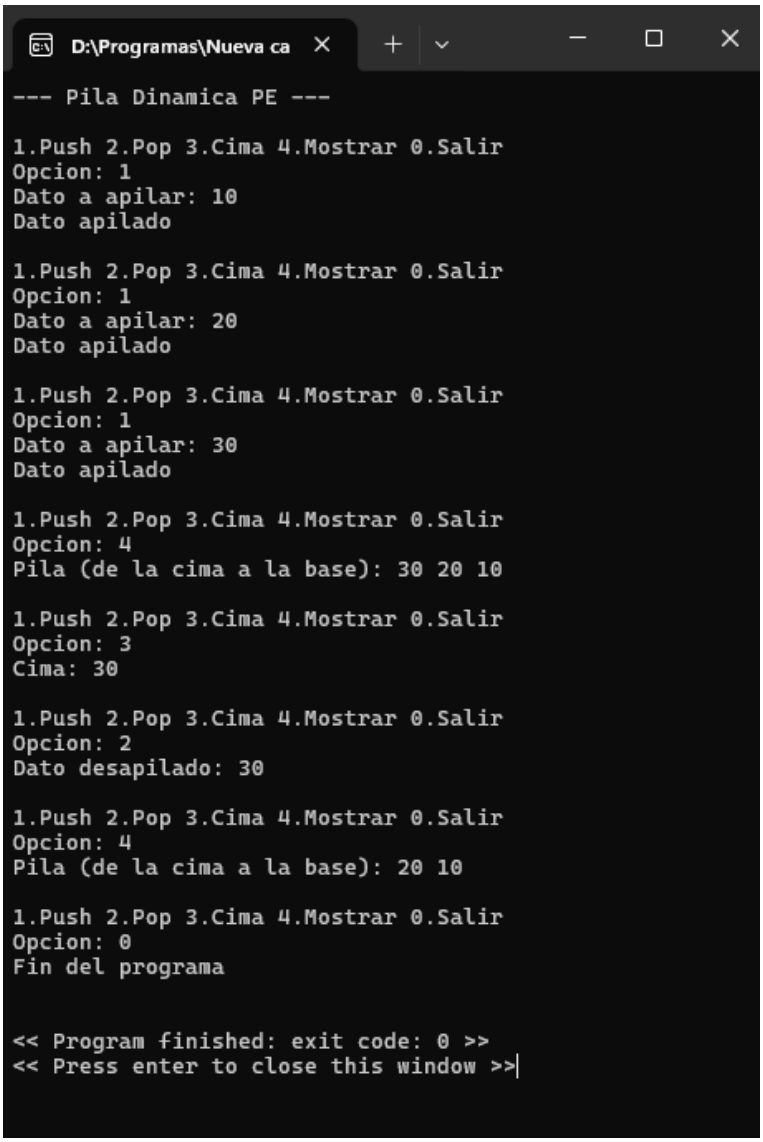
int main() {
    Pila pila;
    iniciar(pila);
    int opcion, dato;

    cout << "--- Pila Dinamica PE ---\n";
    do {
        cout << "\n1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Dato a apilar: ";
            cin >> dato;
            push(pila, dato);
            cout << "Dato apilado\n";
            break;
        case 2:
            if (pop(pila, dato)) cout << "Dato desapilado: " << dato << endl;
            else cout << "Pila vacia: no se puede desapilar\n";
            break;
        case 3:
            if (cima(pila, dato)) cout << "Cima: " << dato << endl;
            else cout << "Pila vacia\n";
            break;
        case 4:
            mostrar(pila);
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);

    liberar(pila);
    return 0;
}

```

Salida


D:\Programas\Nueva ca
--- Pila Dinamica PE ---
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 1
Dato a apilar: 10
Dato apilado
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 1
Dato a apilar: 20
Dato apilado
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 1
Dato a apilar: 30
Dato apilado
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 4
Pila (de la cima a la base): 30 20 10
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 3
Cima: 30
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 2
Dato desapilado: 30
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 4
Pila (de la cima a la base): 20 10
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 0
Fin del programa
<< Program finished: exit code: 0 >>
<< Press enter to close this window >>

Código POO

Ejercicio6_POO.cpp

```cpp
#include <iostream>
using namespace std;

class PilaDinamica {
private:
    struct Nodo {
        int dato;
        Nodo* siguiente;
    };
    Nodo* tope;
public:
    PilaDinamica() { tope = nullptr; }

    ~PilaDinamica() {
        int dato;
        while (pop(dato));
    }

    bool vacia() { return tope == nullptr; }

    void push(int dato) {
        Nodo* nuevo = new Nodo;
        nuevo->dato = dato;
        nuevo->siguiente = tope;
        tope = nuevo;
    }

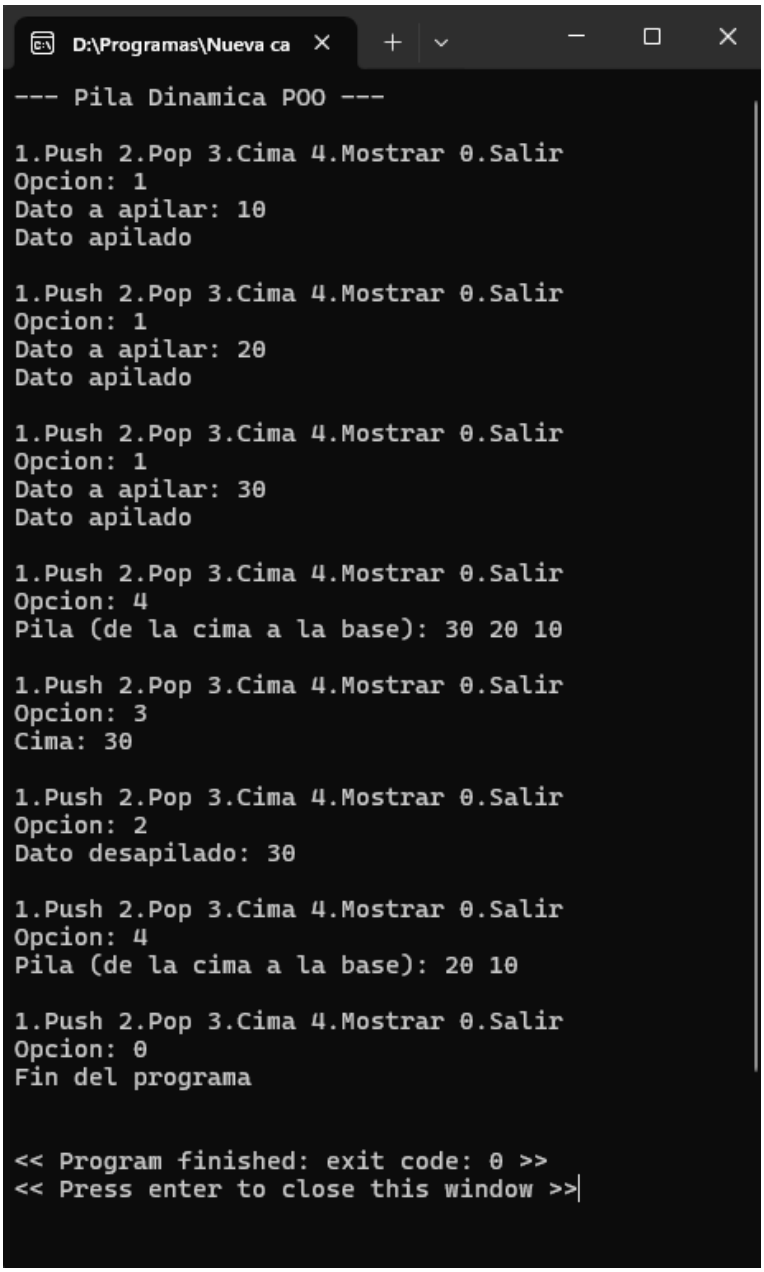
    bool pop(int &dato) {
        if (vacia()) return false;
        Nodo* aux = tope;
        dato = aux->dato;
        tope = aux->siguiente;
        delete aux;
        return true;
    }

    bool cima(int &dato) {
        if (vacia()) return false;
        dato = tope->dato;
        return true;
    }

```

```
43      void mostrar() {
44          if (vacia()) {
45              cout << "La pila esta vacia\n";
46              return;
47          }
48          cout << "Pila (de la cima a la base): ";
49          Nodo* aux = tope;
50          while (aux != nullptr) {
51              cout << aux->dato << " ";
52              aux = aux->siguiente;
53          }
54          cout << endl;
55      }
56  };
57
58  int main() {
59      PilaDinamica pila;
60      int opcion, dato;
61
62      cout << "--- Pila Dinamica POO ---\n";
63      do {
64          cout << "\n1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir\nOpcion: ";
65          cin >> opcion;
66          switch (opcion) {
67          case 1:
68              cout << "Dato a apilar: ";
69              cin >> dato;
70              pila.push(dato);
71              cout << "Dato apilado\n";
72              break;
73          case 2:
74              if (pila.pop(dato)) cout << "Dato desapilado: " << dato << endl;
75              else cout << "Pila vacia: no se puede desapilar\n";
76              break;
77          case 3:
78              if (pila.cima(dato)) cout << "Cima: " << dato << endl;
79              else cout << "Pila vacia\n";
80              break;
81          case 4:
82              pila.mostrar();
83              break;
84          case 0:
85              cout << "Fin del programa\n";
86              break;
87          default:
88              cout << "Opcion invalida\n";
89          }
90      } while (opcion != 0);
91      return 0;
92  }
93
```

Salida



Explicación del Código

- Pila dinámica: la pila se implementa con nodos enlazados. Cada Nodo guarda un dato y un puntero siguiente al mismo tipo de dato (Nodo*), que apunta al nodo que está debajo en la pila. Un puntero tope apunta al nodo de la cima; cuando la pila está vacía, tope vale nullptr.

- Estructura LIFO: el último dato en entrar es el primero en salir. Por eso, después de apilar 10, 20 y 30, el primer dato que se desapila es el 30.

- Memoria dinámica: no existe un tamaño máximo definido. Cada push() reserva un nodo nuevo con new, y cada pop() lo libera con delete, de modo que la pila crece y se reduce según se necesite y no existe el caso de pila llena.

- Push (apilar): crea un nodo nuevo con el dato, hace que su puntero siguiente apunte al tope actual y después actualiza tope para que apunte al nodo nuevo.

- Pop (desapilar): primero verifica que la pila no esté vacía (tope == nullptr). Si tiene datos, guarda el dato del nodo de la cima, mueve tope al nodo siguiente y libera el nodo anterior con delete. Si está vacía, regresa false y el programa avisa que no hay nada que desapilar.

- Cima: devuelve el dato del nodo apuntado por tope sin eliminarlo, por lo que la pila no cambia.

- Mostrar: recorre los nodos con un puntero auxiliar, desde tope hasta llegar a nullptr, para imprimir los datos de la cima a la base.

- Liberación de memoria: al terminar el programa deben liberarse los nodos que aún quedan en la pila. En PE se hace con la función liberar(), que desapila hasta vaciarla; en POO se hace automáticamente con el destructor (~PilaDinamica), que se ejecuta cuando el objeto termina su vida.

- Operador flecha: aux->dato y aux->siguiente acceden a los campos del nodo al que apunta el puntero. Es equivalente a (*aux).dato.

- Parámetros por referencia: en pop() y cima() el dato se devuelve por medio del parámetro &dato, mientras que el valor de retorno (bool) indica si la operación fue exitosa.

- Menú interactivo: el ciclo do-while repite el menú hasta que el usuario elige 0, y el switch ejecuta la operación elegida.

- Enfoque PE: Nodo y Pila son estructuras, y las operaciones (iniciar, push, pop, cima, mostrar, liberar) son funciones globales que reciben la pila por referencia (&).

- Enfoque POO: la clase PilaDinamica encapsula el nodo (como estructura privada) y el puntero tope. El constructor inicializa tope en nullptr, el destructor libera la memoria, y las operaciones son métodos públicos que trabajan directamente sobre los datos del objeto.

- Lo que imprime: con la secuencia de ejemplo, se apilan 10, 20 y 30, se muestra la pila (30 20 10), se consulta la cima (30), se desapila el 30 y se muestra la pila final (20 10). El resultado es el mismo en PE y POO; solo cambia el título “Pila Dinamica PE” / “Pila Dinamica POO”.

- Errores y Soluciones: no hubo errores.

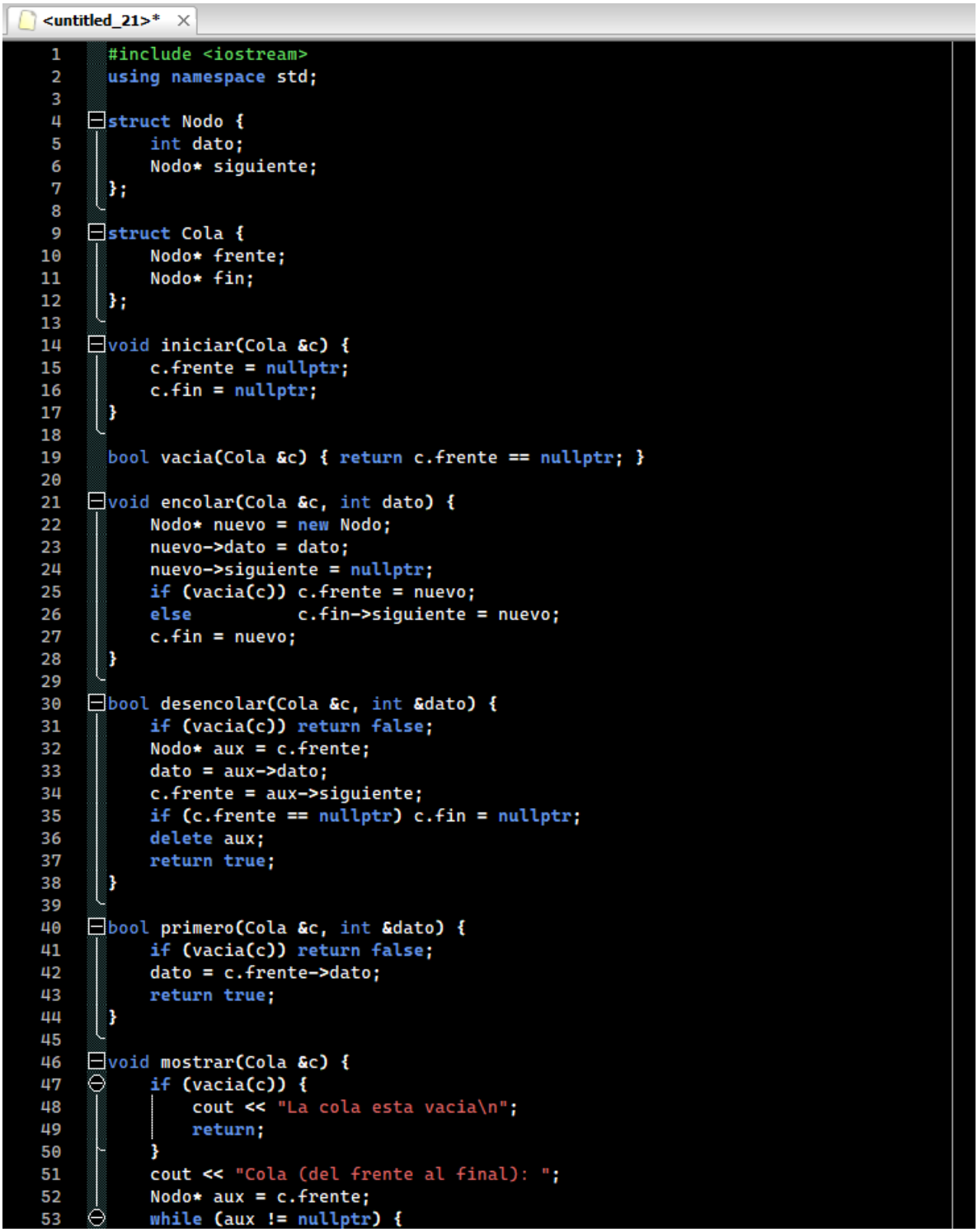
Ejercicio 7 - Cola dinámica por medio de punteros al mismo tipo de dato (PE y POO)

Objetivo del código

Implementar una cola (estructura FIFO, primero en entrar, primero en salir) de manera dinámica por medio de nodos enlazados con punteros al mismo tipo de dato, aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). A diferencia de la cola estática, no tiene un tamaño máximo fijo: la memoria se reserva con new al encolar y se libera con delete al desencolar. El programa permite al usuario, mediante un menú en consola, encolar,

desencolar, consultar el primer elemento y mostrar el contenido de la cola, controlando el caso de cola vacía.

Código PE



```
54          cout << aux->dato << " ";
55          aux = aux->siguiente;
56      }
57      cout << endl;
58  }
59
60  void liberar(Cola &c) {
61      int dato;
62      while (desencolar(c, dato));
63  }
64
65  int main() {
66      Cola cola;
67      iniciar(cola);
68      int opcion, dato;
69
70      cout << "--- Cola Dinamica PE ---\n";
71      do {
72          cout << "\n1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir\nOpcion: ";
73          cin >> opcion;
74          switch (opcion) {
75          case 1:
76              cout << "Dato a encolar: ";
77              cin >> dato;
78              encolar(cola, dato);
79              cout << "Dato encolado\n";
80              break;
81          case 2:
82              if (desencolar(cola, dato)) cout << "Dato desencolado: " << dato << endl;
83              else cout << "Cola vacia: no se puede desencolar\n";
84              break;
85          case 3:
86              if (primero(cola, dato)) cout << "Primero: " << dato << endl;
87              else cout << "Cola vacia\n";
88              break;
89          case 4:
90              mostrar(cola);
91              break;
92          case 0:
93              cout << "Fin del programa\n";
94              break;
95          default:
96              cout << "Opcion invalida\n";
97          }
98      } while (opcion != 0);
99
100     liberar(cola);
101     return 0;
102 }
103
```

Salida


D:\Programas\Nueva ca
--- Cola Dinamica PE ---
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 10
Dato encolado
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 20
Dato encolado
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 30
Dato encolado
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 4
Cola (del frente al final): 10 20 30
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 3
Primero: 10
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 2
Dato desencolado: 10
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 4
Cola (del frente al final): 20 30
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 0
Fin del programa
<< Program finished: exit code: 0 >>
<< Press enter to close this window >>

Código POO

Ejercicio7_POO.cpp

```cpp
#include <iostream>
using namespace std;

class ColaDinamica {
private:
    struct Nodo {
        int dato;
        Nodo* siguiente;
    };
    Nodo* frente;
    Nodo* fin;
public:
    ColaDinamica() {
        frente = nullptr;
        fin = nullptr;
    }

    ~ColaDinamica() {
        int dato;
        while (desencolar(dato));
    }

    bool vacia() { return frente == nullptr; }

    void encolar(int dato) {
        Nodo* nuevo = new Nodo;
        nuevo->dato = dato;
        nuevo->siguiente = nullptr;
        if (vacia()) frente = nuevo;
        else         fin->siguiente = nuevo;
        fin = nuevo;
    }

    bool desencolar(int &dato) {
        if (vacia()) return false;
        Nodo* aux = frente;
        dato = aux->dato;
        frente = aux->siguiente;
        if (frente == nullptr) fin = nullptr;
        delete aux;
        return true;
    }

    bool primero(int &dato) {
        if (vacia()) return false;
        dato = frente->dato;
        return true;
    }

```

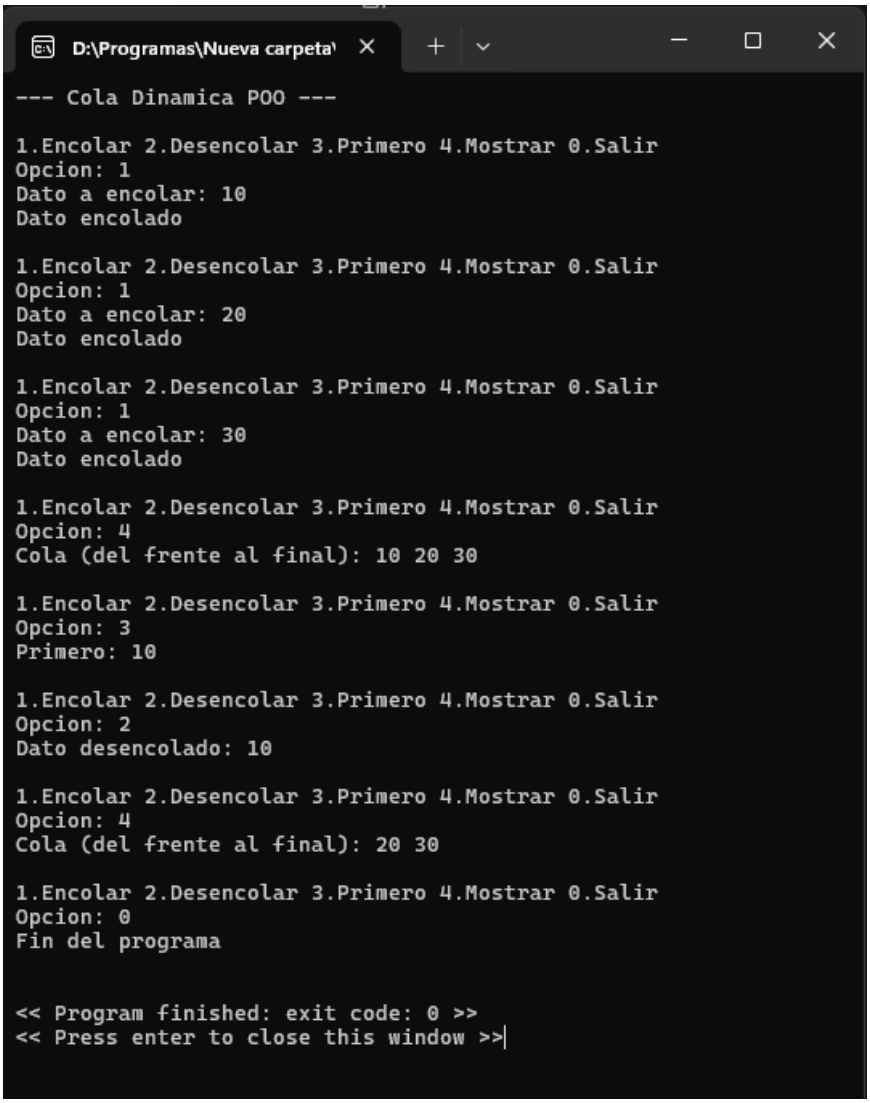
```
    void mostrar() {
        if (vacia()) {
            cout << "La cola esta vacia\n";
            return;
        }
        cout << "Cola (del frente al final): ";
        Nodo* aux = frente;
        while (aux != nullptr) {
            cout << aux->dato << " ";
            aux = aux->siguiente;
        }
        cout << endl;
    }
};

int main() {
    ColaDinamica cola;
    int opcion, dato;

    cout << "--- Cola Dinamica POO ---\n";
    do {
        cout << "\n1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Dato a encolar: ";
            cin >> dato;
            cola.encolar(dato);
            cout << "Dato encolado\n";
            break;
        case 2:
            if (cola.desencolar(dato)) cout << "Dato desencolado: " << dato << endl;
            else cout << "Cola vacia: no se puede desencolar\n";
            break;
        case 3:
            if (cola.primero(dato)) cout << "Primero: " << dato << endl;
            else cout << "Cola vacia\n";
            break;
        case 4:
            cola.mostrar();
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}

```

Salida



Explicación del Código

- Cola dinámica: la cola se implementa con nodos enlazados. Cada Nodo guarda un dato y un puntero siguiente al mismo tipo de dato (Nodo*), que apunta al nodo que está detrás en la cola. Dos punteros controlan los extremos: frente apunta al primer nodo (el que sale) y fin apunta al último nodo (donde se agrega). Cuando la cola está vacía, ambos valen nullptr.

- Estructura FIFO: el primer dato en entrar es el primero en salir. Por eso, después de encolar 10, 20 y 30, el primer dato que se desencola es el 10.

- Memoria dinámica: no existe un tamaño máximo definido. Cada encolar() reserva un nodo nuevo con new, y cada desencolar() lo libera con delete, de modo que la cola crece y se reduce según se necesite y no existe el caso de cola llena.

- Encolar: crea un nodo nuevo con el dato y siguiente en nullptr. Si la cola está vacía, el nodo nuevo es también el frente; si no, el último nodo actual se enlaza al nodo nuevo. Después, fin se actualiza para apuntar al nodo nuevo.

- Desencolar: primero verifica que la cola no esté vacía (frente == nullptr). Si tiene datos, guarda el dato del primer nodo, mueve frente al nodo siguiente y libera el nodo anterior con delete. Si la cola queda vacía, fin también se regresa a nullptr. Si estaba vacía, regresa false y el programa avisa que no hay nada que desencolar.

- Primero: devuelve el dato del nodo apuntado por frente sin eliminarlo, por lo que la cola no cambia.

- Mostrar: recorre los nodos con un puntero auxiliar, desde frente hasta llegar a nullptr, para imprimir los datos del frente al final.

- Liberación de memoria: al terminar el programa deben liberarse los nodos que aún quedan en la cola. En PE se hace con la función liberar(), que desencola hasta vaciarla; en POO se hace automáticamente con el destructor (~ColaDinamica), que se ejecuta cuando el objeto termina su vida.

- Operador flecha: aux->dato y aux->siguiente acceden a los campos del nodo al que apunta el puntero. Es equivalente a (*aux).dato.

- Parámetros por referencia: en desencolar() y primero() el dato se devuelve por medio del parámetro &dato, mientras que el valor de retorno (bool) indica si la operación fue exitosa.

- Menú interactivo: el ciclo do-while repite el menú hasta que el usuario elige 0, y el switch ejecuta la operación elegida.

- Enfoque PE: Nodo y Cola son estructuras, y las operaciones (iniciar, encolar, desencolar, primero, mostrar, liberar) son funciones globales que reciben la cola por referencia (&).

- Enfoque POO: la clase ColaDinamica encapsula el nodo (como estructura privada) y los punteros frente y fin. El constructor los inicializa en nullptr, el destructor libera la memoria, y las operaciones son métodos públicos que trabajan directamente sobre los datos del objeto.

- Lo que imprime: con la secuencia de ejemplo, se encolan 10, 20 y 30, se muestra la cola (10 20 30), se consulta el primero (10), se desencola el 10 y se muestra la cola final (20 30). El resultado es el mismo en PE y POO; solo cambia el título “Cola Dinamica PE” / “Cola Dinamica POO”.

- Errores y Soluciones: no hubo errores.

Ejercicio 7 bis - Manejo de una lista de manera dinámica por medio de punteros al mismo tipo de dato (PE y POO)

Objetivo del código

Implementar una lista de manera dinámica por medio de nodos enlazados con punteros al mismo tipo de dato, aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). A diferencia de la lista estática, no tiene un tamaño máximo fijo: la memoria se reserva con new al insertar y se libera con delete al eliminar. El programa permite al usuario, mediante un menú en consola, insertar, eliminar, buscar y mostrar los datos de la lista en cualquier posición, controlando los casos de lista vacía y posición inválida.

Código PE

Ejercicio7b_PE.cpp

```cpp
#include <iostream>
using namespace std;

struct Nodo {
    int dato;
    Nodo* siguiente;
};

struct Lista {
    Nodo* inicio;
    int cantidad;
};

void iniciar(Lista &l) {
    l.inicio = nullptr;
    l.cantidad = 0;
}

bool vacia(Lista &l) { return l.inicio == nullptr; }

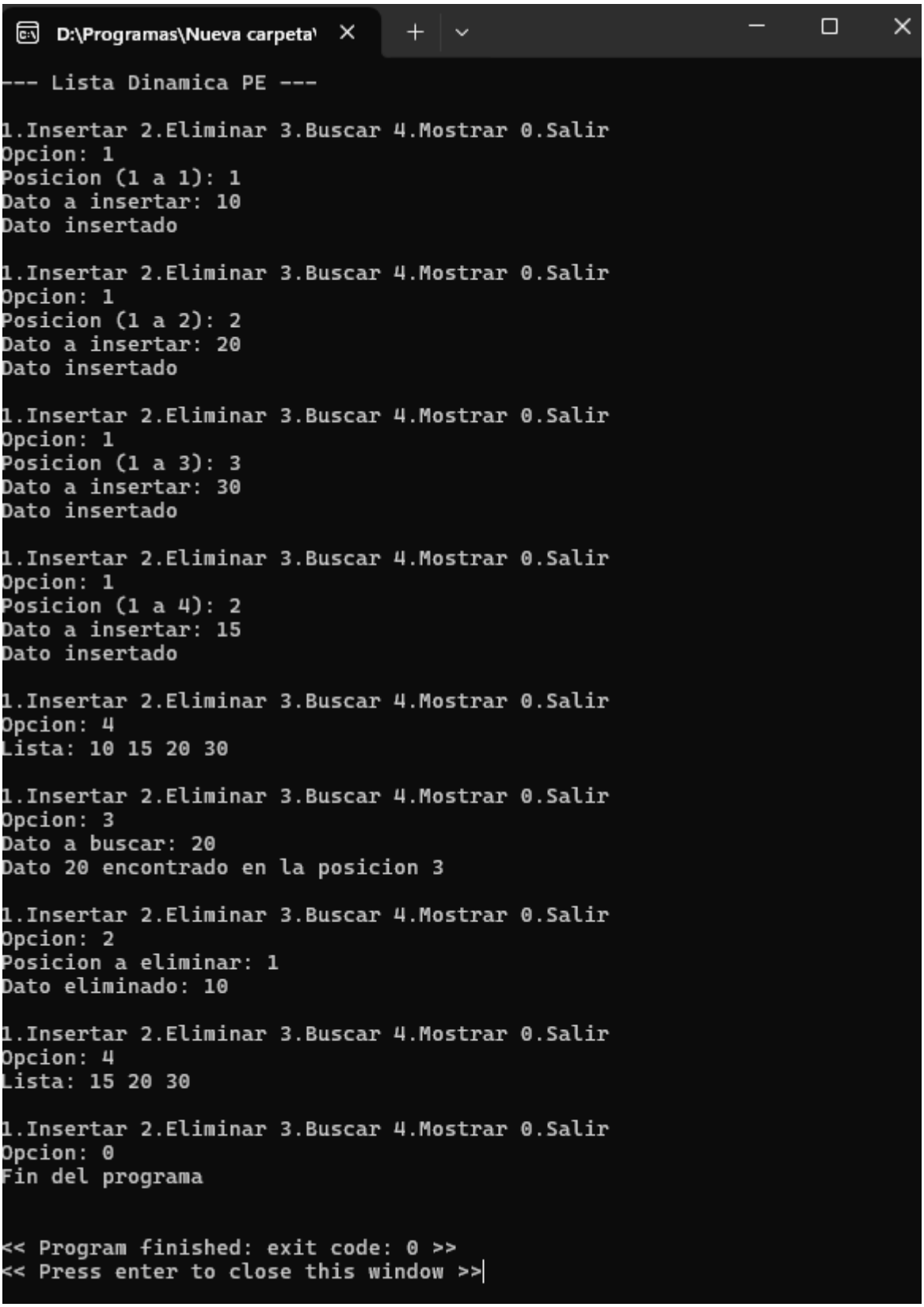
bool insertar(Lista &l, int pos, int dato) {
    if (pos < 1 || pos > l.cantidad + 1) return false;
    Nodo* nuevo = new Nodo;
    nuevo->dato = dato;
    if (pos == 1) {
        nuevo->siguiente = l.inicio;
        l.inicio = nuevo;
    } else {
        Nodo* ant = l.inicio;
        for (int i = 1; i < pos - 1; i++) ant = ant->siguiente;
        nuevo->siguiente = ant->siguiente;
        ant->siguiente = nuevo;
    }
    l.cantidad++;
    return true;
}

bool eliminar(Lista &l, int pos, int &dato) {
    if (pos < 1 || pos > l.cantidad) return false;
    Nodo* aux;
    if (pos == 1) {
        aux = l.inicio;
        l.inicio = aux->siguiente;
    } else {
        Nodo* ant = l.inicio;
        for (int i = 1; i < pos - 1; i++) ant = ant->siguiente;
        aux = ant->siguiente;
        ant->siguiente = aux->siguiente;
    }
    dato = aux->dato;
    delete aux;
    l.cantidad--;
    return true;
}
```

```
55
56  int buscar(Lista &l, int dato) {
57      Nodo* aux = l.inicio;
58      int pos = 1;
59      while (aux != nullptr) {
60          if (aux->dato == dato) return pos;
61          aux = aux->siguiente;
62          pos++;
63      }
64      return 0;
65  }
66
67  void mostrar(Lista &l) {
68      if (vacia(l)) {
69          cout << "La lista esta vacia\n";
70          return;
71      }
72      cout << "Lista: ";
73      Nodo* aux = l.inicio;
74      while (aux != nullptr) {
75          cout << aux->dato << " ";
76          aux = aux->siguiente;
77      }
78      cout << endl;
79  }
80
81  void liberar(Lista &l) {
82      int dato;
83      while (eliminar(l, 1, dato));
84  }
85
86  int main() {
87      Lista lista;
88      iniciar(lista);
89      int opcion, dato, pos;
90
91      cout << "--- Lista Dinamica PE ---\n";
92      do {
93          cout << "\n1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir\nOpcion: ";
94          cin >> opcion;
95          switch (opcion) {
96          case 1:
97              cout << "Posicion (1 a " << lista.cantidad + 1 << "): ";
98              cin >> pos;
99              cout << "Dato a insertar: ";
100             cin >> dato;
101             if (insertar(lista, pos, dato)) cout << "Dato insertado\n";
102             else cout << "Posicion invalida\n";
103             break;
104         case 2:
105             cout << "Posicion a eliminar: ";
106             cin >> pos;
107             if (eliminar(lista, pos, dato)) cout << "Dato eliminado: " << dato << endl;
108             else cout << "Lista vacia o posicion invalida\n";
```

```
109             break;
110         case 3:
111             cout << "Dato a buscar: ";
112             cin >> dato;
113             pos = buscar(lista, dato);
114             if (pos > 0) cout << "Dato " << dato << " encontrado en la posicion " << pos << endl;
115             else cout << "Dato no encontrado\n";
116             break;
117         case 4:
118             mostrar(lista);
119             break;
120         case 0:
121             cout << "Fin del programa\n";
122             break;
123         default:
124             cout << "Opcion invalida\n";
125         }
126     } while (opcion != 0);
127
128     liberar(lista);
129     return 0;
130 }
131
```

Salida


D:\Programas\Nueva carpeta
--- Lista Dinamica PE ---
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 1): 1
Dato a insertar: 10
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 2): 2
Dato a insertar: 20
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 3): 3
Dato a insertar: 30
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 4): 2
Dato a insertar: 15
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 4
Lista: 10 15 20 30
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 3
Dato a buscar: 20
Dato 20 encontrado en la posicion 3
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 2
Posicion a eliminar: 1
Dato eliminado: 10
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 4
Lista: 15 20 30
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 0
Fin del programa
<< Program finished: exit code: 0 >>
<< Press enter to close this window >>

Código POO

Ejercicio7b_POO.cpp

```cpp
#include <iostream>
using namespace std;

class ListaDinamica {
private:
    struct Nodo {
        int dato;
        Nodo* siguiente;
    };
    Nodo* inicio;
    int cantidad;
public:
    ListaDinamica() {
        inicio = nullptr;
        cantidad = 0;
    }

    ~ListaDinamica() {
        int dato;
        while (eliminar(1, dato));
    }

    int getCantidad() { return cantidad; }
    bool vacia()      { return inicio == nullptr; }

    bool insertar(int pos, int dato) {
        if (pos < 1 || pos > cantidad + 1) return false;
        Nodo* nuevo = new Nodo;
        nuevo->dato = dato;
        if (pos == 1) {
            nuevo->siguiente = inicio;
            inicio = nuevo;
        } else {
            Nodo* ant = inicio;
            for (int i = 1; i < pos - 1; i++) ant = ant->siguiente;
            nuevo->siguiente = ant->siguiente;
            ant->siguiente = nuevo;
        }
        cantidad++;
        return true;
    }

    bool eliminar(int pos, int &dato) {
        if (pos < 1 || pos > cantidad) return false;
        Nodo* aux;
        if (pos == 1) {
            aux = inicio;
            inicio = aux->siguiente;
        } else {
            Nodo* ant = inicio;
            for (int i = 1; i < pos - 1; i++) ant = ant->siguiente;
            aux = ant->siguiente;
            ant->siguiente = aux->siguiente;
```

```
        }
        dato = aux->dato;
        delete aux;
        cantidad--;
        return true;
    }

    int buscar(int dato) {
        Nodo* aux = inicio;
        int pos = 1;
        while (aux != nullptr) {
            if (aux->dato == dato) return pos;
            aux = aux->siguiente;
            pos++;
        }
        return 0;
    }

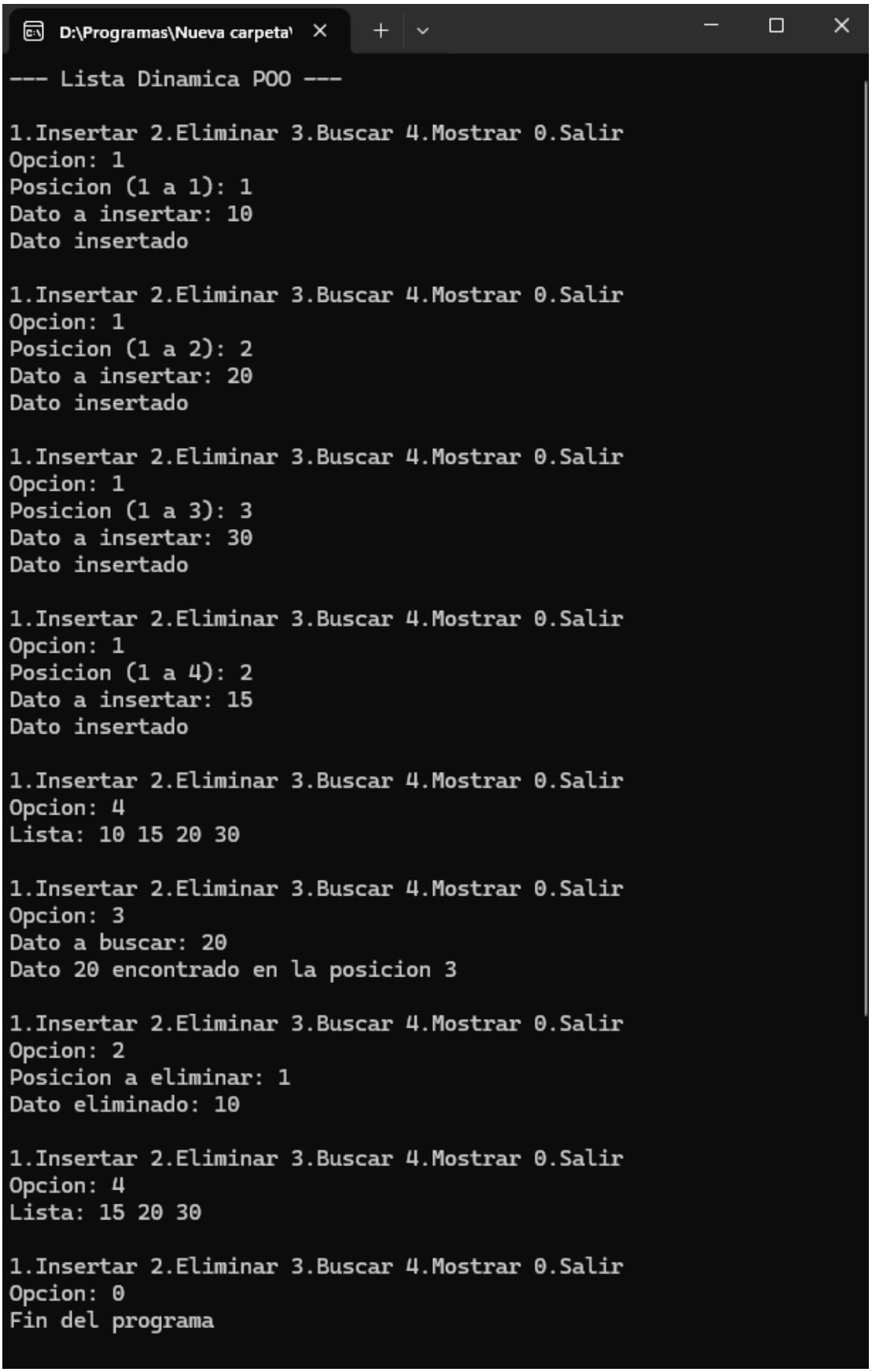
    void mostrar() {
        if (vacia()) {
            cout << "La lista esta vacia\n";
            return;
        }
        cout << "Lista: ";
        Nodo* aux = inicio;
        while (aux != nullptr) {
            cout << aux->dato << " ";
            aux = aux->siguiente;
        }
        cout << endl;
    }
};

int main() {
    ListaDinamica lista;
    int opcion, dato, pos;

    cout << "--- Lista Dinamica POO ---\n";
    do {
        cout << "\n1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Posicion (1 a " << lista.getCantidad() + 1 << "): ";
            cin >> pos;
            cout << "Dato a insertar: ";
            cin >> dato;
            if (lista.insertar(pos, dato)) cout << "Dato insertado\n";
            else cout << "Posicion invalida\n";
            break;
        case 2:
            cout << "Posicion a eliminar: ";
```

```
106             cin >> pos;
107             if (lista.eliminar(pos, dato)) cout << "Dato eliminado: " << dato << endl;
108             else cout << "Lista vacia o posicion invalida\n";
109             break;
110         case 3:
111             cout << "Dato a buscar: ";
112             cin >> dato;
113             pos = lista.buscar(dato);
114             if (pos > 0) cout << "Dato " << dato << " encontrado en la posicion " << pos << endl;
115             else cout << "Dato no encontrado\n";
116             break;
117         case 4:
118             lista.mostrar();
119             break;
120         case 0:
121             cout << "Fin del programa\n";
122             break;
123         default:
124             cout << "Opcion invalida\n";
125         }
126     } while (opcion != 0);
127     return 0;
128 }
129
```

Salida


D:\Programas\Nueva carpeta
--- Lista Dinamica POO ---
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 1): 1
Dato a insertar: 10
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 2): 2
Dato a insertar: 20
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 3): 3
Dato a insertar: 30
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 4): 2
Dato a insertar: 15
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 4
Lista: 10 15 20 30
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 3
Dato a buscar: 20
Dato 20 encontrado en la posicion 3
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 2
Posicion a eliminar: 1
Dato eliminado: 10
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 4
Lista: 15 20 30
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 0
Fin del programa

Explicación del Código

- Lista dinámica: la lista se implementa con nodos enlazados. Cada Nodo guarda un dato y un puntero siguiente al mismo tipo de dato (Nodo*), que apunta al nodo que está después en la lista. Un puntero inicio apunta al primer nodo y una variable cantidad guarda cuántos nodos hay. Cuando la lista está vacía, inicio vale nullptr.

- Memoria dinámica: no existe un tamaño máximo definido. Cada insertar() reserva un nodo nuevo con new, y cada eliminar() lo libera con delete, por lo que no existe el caso de lista llena y no hay que recorrer los datos para hacer espacio, como ocurre en la lista estática.

- Posiciones: el usuario trabaja con posiciones desde 1, y el programa las valida con cantidad (de 1 a cantidad + 1 para insertar, y de 1 a cantidad para eliminar).

- Insertar: crea un nodo nuevo con el dato. Si la posición es 1, el nodo nuevo apunta al antiguo primer nodo y pasa a ser el inicio. Si es otra posición, se recorre la lista hasta el nodo anterior (ant), el nodo nuevo apunta al nodo que seguía a ant y después ant apunta al nodo nuevo. En el ejemplo, al insertar 15 en la posición 2, el nodo 10 queda apuntando al 15 y el 15 apuntando al 20.

- Eliminar: si la posición es 1, inicio avanza al segundo nodo. Si es otra posición, se recorre hasta el nodo anterior (ant) y se enlaza con el nodo que sigue al que se elimina. Después se guarda el dato, se libera el nodo con delete y se disminuye cantidad. En el ejemplo, al eliminar la posición 1 se obtiene el 10 y la lista queda 15 20 30.

- Buscar: recorre los nodos con un puntero auxiliar comparando cada dato con el buscado. Regresa su posición (a partir de 1) si lo encuentra, o 0 si no existe.

- Mostrar: recorre los nodos con un puntero auxiliar, desde inicio hasta llegar a nullptr, para imprimir todos los datos en orden.

- Liberación de memoria: al terminar el programa deben liberarse los nodos que aún quedan en la lista. En PE se hace con la función liberar(), que elimina el primer nodo hasta vaciarla; en POO se hace automáticamente con el destructor (~ListaDinamica), que se ejecuta cuando el objeto termina su vida.

- Operador flecha: aux->dato y aux->siguiente acceden a los campos del nodo al que apunta el puntero. Es equivalente a (*aux).dato.

- Parámetros por referencia: en eliminar() el dato se devuelve por medio del parámetro &dato, mientras que el valor de retorno (bool) indica si la operación fue exitosa.

- Menú interactivo: el ciclo do-while repite el menú hasta que el usuario elige 0, y el switch ejecuta la operación elegida.

- Enfoque PE: Nodo y Lista son estructuras, y las operaciones (iniciar, insertar, eliminar, buscar, mostrar, liberar) son funciones globales que reciben la lista por referencia (&).

- Enfoque POO: la clase ListaDinamica encapsula el nodo (como estructura privada), el puntero inicio y la cantidad. El constructor los inicializa, el destructor libera la memoria, y las operaciones son métodos públicos que trabajan directamente sobre los datos del objeto.

- Lo que imprime: con la secuencia de ejemplo, se insertan 10, 20 y 30, se inserta 15 en la posición 2, se muestra la lista (10 15 20 30), se busca el 20 (posición 3), se elimina la posición 1 (dato 10) y se muestra la lista final (15 20 30). El resultado es el mismo en PE y POO; solo cambia el título “Lista Dinamica PE” / “Lista Dinamica POO”.

- Errores y Soluciones: no hubo errores.

Ejercicio 8 - Pila dinámica por medio de las librerías (PE y POO)

Objetivo del código

Implementar una pila (estructura LIFO, último en entrar, primero en salir) de manera dinámica utilizando la clase stack de la biblioteca estándar de C++ (STL), en lugar de programar los nodos y punteros manualmente, aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). El programa permite al usuario, mediante un menú en consola, apilar (push), desapilar (pop), consultar la cima y mostrar el contenido de la pila, controlando el caso de pila vacía.

Código PE

Ejercicio8_PE.cpp

```cpp
#include <iostream>
#include <stack>
using namespace std;

void push(stack<int> &p, int dato) {
    p.push(dato);
}

bool pop(stack<int> &p, int &dato) {
    if (p.empty()) return false;
    dato = p.top();
    p.pop();
    return true;
}

bool cima(stack<int> &p, int &dato) {
    if (p.empty()) return false;
    dato = p.top();
    return true;
}

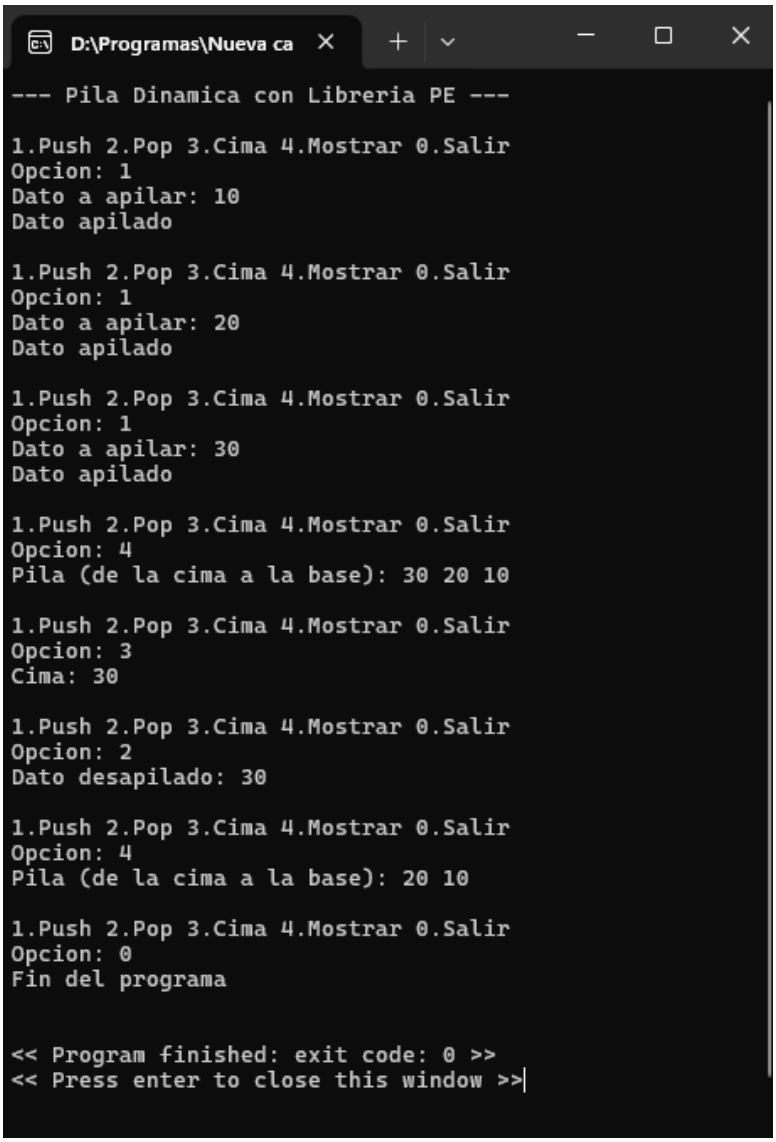
// Recibe una copia de la pila para poder vaciarla sin afectar la original
void mostrar(stack<int> copia) {
    if (copia.empty()) {
        cout << "La pila esta vacia\n";
        return;
    }
    cout << "Pila (de la cima a la base): ";
    while (!copia.empty()) {
        cout << copia.top() << " ";
        copia.pop();
    }
    cout << endl;
}

```

```cpp
int main() {
    stack<int> pila;
    int opcion, dato;

    cout << "--- Pila Dinamica con Libreria PE ---\n";
    do {
        cout << "\n1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Dato a apilar: ";
            cin >> dato;
            push(pila, dato);
            cout << "Dato apilado\n";
            break;
        case 2:
            if (pop(pila, dato)) cout << "Dato desapilado: " << dato << endl;
            else cout << "Pila vacia: no se puede desapilar\n";
            break;
        case 3:
            if (cima(pila, dato)) cout << "Cima: " << dato << endl;
            else cout << "Pila vacia\n";
            break;
        case 4:
            mostrar(pila);
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}
```

Salida


D:\Programas\Nueva ca
--- Pila Dinamica con Libreria PE ---
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 1
Dato a apilar: 10
Dato apilado
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 1
Dato a apilar: 20
Dato apilado
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 1
Dato a apilar: 30
Dato apilado
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 4
Pila (de la cima a la base): 30 20 10
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 3
Cima: 30
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 2
Dato desapilado: 30
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 4
Pila (de la cima a la base): 20 10
1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir
Opcion: 0
Fin del programa
<< Program finished: exit code: 0 >>
<< Press enter to close this window >>

Código POO

Ejercicio8_PE.cpp | <untitled_26>*

```cpp
#include <iostream>
#include <stack>
using namespace std;

class PilaLibreria {
private:
    stack<int> pila;
public:
    bool vacia() { return pila.empty(); }

    void push(int dato) {
        pila.push(dato);
    }

    bool pop(int &dato) {
        if (vacia()) return false;
        dato = pila.top();
        pila.pop();
        return true;
    }

    bool cima(int &dato) {
        if (vacia()) return false;
        dato = pila.top();
        return true;
    }

    void mostrar() {
        if (vacia()) {
            cout << "La pila esta vacia\n";
            return;
        }
        stack<int> copia = pila;
        cout << "Pila (de la cima a la base): ";
        while (!copia.empty()) {
            cout << copia.top() << " ";
            copia.pop();
        }
        cout << endl;
    }
};

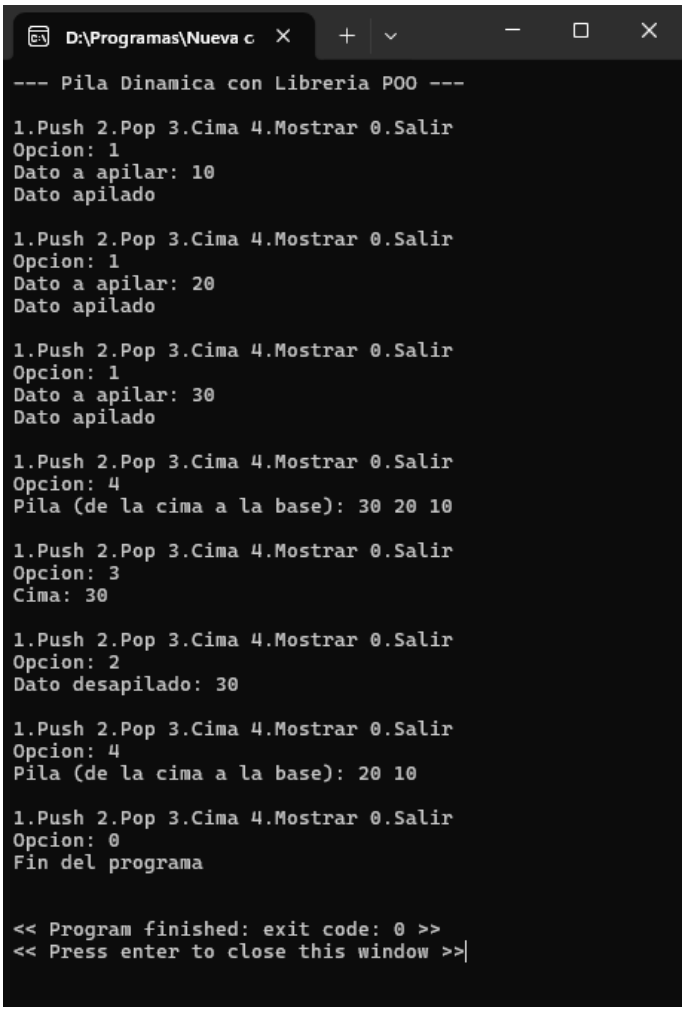
```

```cpp
int main() {
    PilaLibreria pila;
    int opcion, dato;

    cout << "--- Pila Dinamica con Libreria POO ---\n";
    do {
        cout << "\n1.Push 2.Pop 3.Cima 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Dato a apilar: ";
            cin >> dato;
            pila.push(dato);
            cout << "Dato apilado\n";
            break;
        case 2:
            if (pila.pop(dato)) cout << "Dato desapilado: " << dato << endl;
            else cout << "Pila vacia: no se puede desapilar\n";
            break;
        case 3:
            if (pila.cima(dato)) cout << "Cima: " << dato << endl;
            else cout << "Pila vacia\n";
            break;
        case 4:
            pila.mostrar();
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}

```

Salida



Explicación del Código

- Pila con librería: en lugar de crear nodos y punteros manualmente, se utiliza la clase stack de la biblioteca estándar (#include <stack>). La clase administra internamente la memoria de forma dinámica, por lo que la pila crece y se reduce según se necesite y no existe un tamaño máximo definido.

- Estructura LIFO: el último dato en entrar es el primero en salir. Por eso, después de apilar 10, 20 y 30, el primer dato que se desapila es el 30.

- Plantilla stack<int>: el tipo entre los símbolos < > indica qué tipo de dato guarda la pila. Con stack<int> la pila almacena números enteros.

- Métodos de la librería: push() agrega un dato en la cima, pop() elimina el dato de la cima sin devolverlo, top() devuelve el dato de la cima sin eliminarlo y empty() indica si la pila está vacía.

- Pop (desapilar): primero verifica que la pila no esté vacía con empty(). Si tiene datos, lee el dato con top() y después lo elimina con pop(). Si está vacía, regresa false y el programa avisa que no hay nada que desapilar.

- Cima: devuelve el dato con top() sin eliminarlo, por lo que la pila no cambia.

- Mostrar: la clase stack no permite recorrer sus elementos, solo acceder a la cima. Por eso se trabaja con una copia de la pila (copia) que se va vaciando con top() y pop() para imprimir los datos de la cima a la base, sin modificar la pila original.

- Liberación de memoria: no es necesario usar new ni delete; la librería libera la memoria automáticamente cuando la pila se destruye.

- Parámetros por referencia: en pop() y cima() el dato se devuelve por medio del parámetro &dato, mientras que el valor de retorno (bool) indica si la operación fue exitosa. En push() y pop() de PE la pila se recibe por referencia (&) para trabajar sobre la pila original.

- Menú interactivo: el ciclo do-while repite el menú hasta que el usuario elige 0, y el switch ejecuta la operación elegida.

- Enfoque PE: la pila es una variable stack<int> y las operaciones (push, pop, cima, mostrar) son funciones globales que la reciben como parámetro.

- Enfoque POO: la clase PilaLibreria encapsula el objeto stack<int> como atributo privado y expone las operaciones como métodos públicos. No necesita constructor ni destructor propios, porque la librería se encarga de iniciar y liberar la pila.

- Lo que imprime: con la secuencia de ejemplo, se apilan 10, 20 y 30, se muestra la pila (30 20 10), se consulta la cima (30), se desapila el 30 y se muestra la pila final (20 10). El resultado es el mismo en PE y POO; solo cambia el título “Pila Dinamica con Libreria PE” / “Pila Dinamica con Libreria POO”.

- Errores y Soluciones: no hubo errores.

Ejercicio 9 - Cola dinámica por medio de las librerías (PE y POO)

Objetivo del código

Implementar una cola (estructura FIFO, primero en entrar, primero en salir) de manera dinámica utilizando la clase queue de la biblioteca estándar de C++ (STL), en lugar de programar los nodos y punteros manualmente, aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). El programa permite al usuario, mediante un menú en consola, encolar, desencolar, consultar el primer elemento y mostrar el contenido de la cola, controlando el caso de cola vacía.

Código PE

Ejercicio9_PE.cpp

```cpp
#include <iostream>
#include <queue>
using namespace std;

void encolar(queue<int> &c, int dato) {
    c.push(dato);
}

bool desencolar(queue<int> &c, int &dato) {
    if (c.empty()) return false;
    dato = c.front();
    c.pop();
    return true;
}

bool primero(queue<int> &c, int &dato) {
    if (c.empty()) return false;
    dato = c.front();
    return true;
}

// Recibe una copia de la cola para poder vaciarla sin afectar la original
void mostrar(queue<int> copia) {
    if (copia.empty()) {
        cout << "La cola esta vacia\n";
        return;
    }
    cout << "Cola (del frente al final): ";
    while (!copia.empty()) {
        cout << copia.front() << " ";
        copia.pop();
    }
    cout << endl;
}

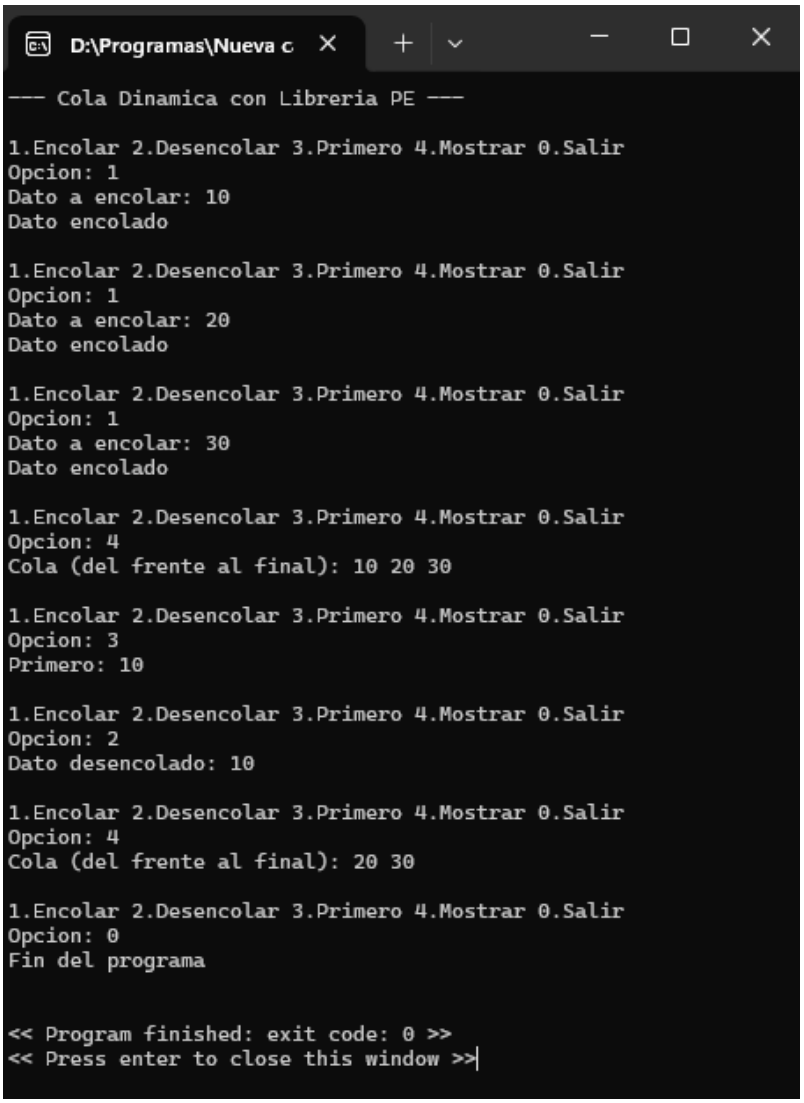
```

```cpp
int main() {
    queue<int> cola;
    int opcion, dato;

    cout << "--- Cola Dinamica con Libreria PE ---\n";
    do {
        cout << "\n1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Dato a encolar: ";
            cin >> dato;
            encolar(cola, dato);
            cout << "Dato encolado\n";
            break;
        case 2:
            if (desencolar(cola, dato)) cout << "Dato desencolado: " << dato << endl;
            else cout << "Cola vacia: no se puede desencolar\n";
            break;
        case 3:
            if (primero(cola, dato)) cout << "Primero: " << dato << endl;
            else cout << "Cola vacia\n";
            break;
        case 4:
            mostrar(cola);
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}

```

Salida


D:\Programas\Nueva c
--- Cola Dinamica con Libreria PE ---
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 10
Dato encolado
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 20
Dato encolado
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 1
Dato a encolar: 30
Dato encolado
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 4
Cola (del frente al final): 10 20 30
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 3
Primero: 10
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 2
Dato desencolado: 10
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 4
Cola (del frente al final): 20 30
1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir
Opcion: 0
Fin del programa
<< Program finished: exit code: 0 >>
<< Press enter to close this window >>

Código POO

Ejercicio9_POO.cpp

```cpp
#include <iostream>
#include <queue>
using namespace std;

class ColaLibreria {
private:
    queue<int> cola;
public:
    bool vacia() { return cola.empty(); }

    void encolar(int dato) {
        cola.push(dato);
    }

    bool desencolar(int &dato) {
        if (vacia()) return false;
        dato = cola.front();
        cola.pop();
        return true;
    }

    bool primero(int &dato) {
        if (vacia()) return false;
        dato = cola.front();
        return true;
    }

    void mostrar() {
        if (vacia()) {
            cout << "La cola esta vacia\n";
            return;
        }
        queue<int> copia = cola;
        cout << "Cola (del frente al final): ";
        while (!copia.empty()) {
            cout << copia.front() << " ";
            copia.pop();
        }
        cout << endl;
    }
};

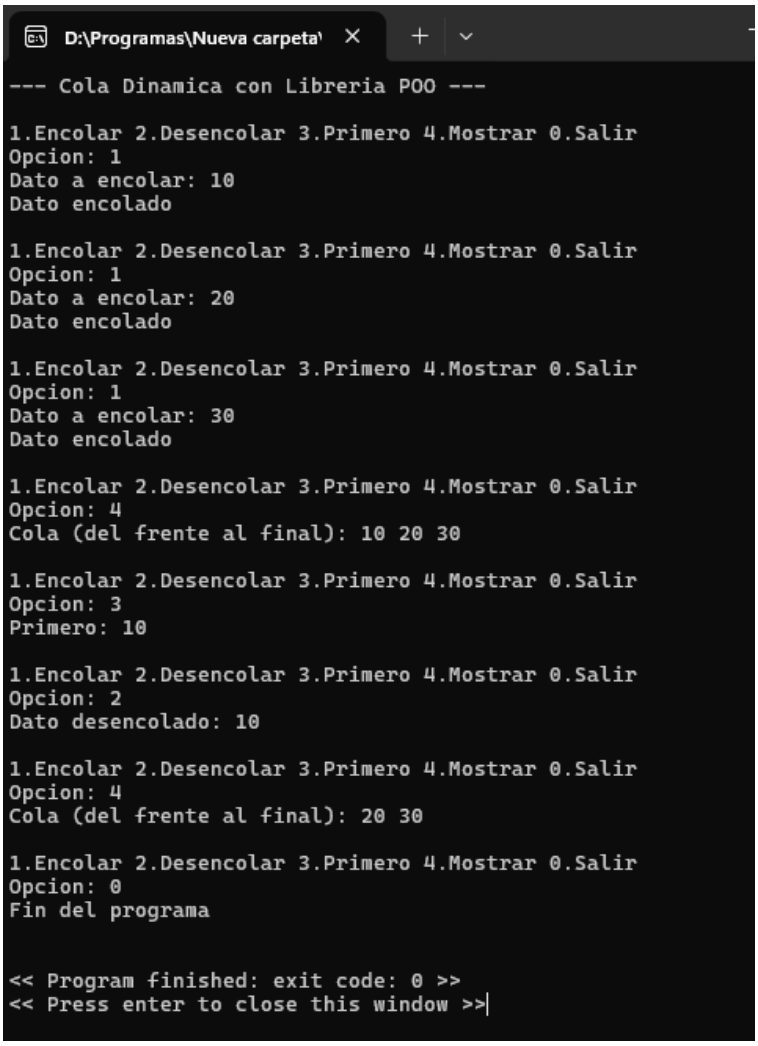
```

```cpp
int main() {
    ColaLibreria cola;
    int opcion, dato;

    cout << "--- Cola Dinamica con Libreria POO ---\n";
    do {
        cout << "\n1.Encolar 2.Desencolar 3.Primero 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Dato a encolar: ";
            cin >> dato;
            cola.encolar(dato);
            cout << "Dato encolado\n";
            break;
        case 2:
            if (cola.desencolar(dato)) cout << "Dato desencolado: " << dato << endl;
            else cout << "Cola vacia: no se puede desencolar\n";
            break;
        case 3:
            if (cola.primero(dato)) cout << "Primero: " << dato << endl;
            else cout << "Cola vacia\n";
            break;
        case 4:
            cola.mostrar();
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}

```

Salida



Explicación del Código

- Cola con librería: en lugar de crear nodos y punteros manualmente, se utiliza la clase queue de la biblioteca estándar (#include <queue>). La clase administra internamente la memoria de forma dinámica, por lo que la cola crece y se reduce según se necesite y no existe un tamaño máximo definido.

- Estructura FIFO: el primer dato en entrar es el primero en salir. Por eso, después de encolar 10, 20 y 30, el primer dato que se desencola es el 10.

- Plantilla queue<int>: el tipo entre los símbolos < > indica qué tipo de dato guarda la cola. Con queue<int> la cola almacena números enteros.

- Métodos de la librería: push() agrega un dato al final de la cola, pop() elimina el dato del frente sin devolverlo, front() devuelve el dato del frente sin eliminarlo y empty() indica si la cola está vacía.

- Desencolar: primero verifica que la cola no esté vacía con empty(). Si tiene datos, lee el dato con front() y después lo elimina con pop(). Si está vacía, regresa false y el programa avisa que no hay nada que desencolar.

- Primero: devuelve el dato con front() sin eliminarlo, por lo que la cola no cambia.

- Mostrar: la clase queue no permite recorrer sus elementos, solo acceder al frente. Por eso se trabaja con una copia de la cola (copia) que se va vaciando con front() y pop() para imprimir los datos del frente al final, sin modificar la cola original.

- Liberación de memoria: no es necesario usar new ni delete; la librería libera la memoria automáticamente cuando la cola se destruye.

- Parámetros por referencia: en desencolar() y primero() el dato se devuelve por medio del parámetro &dato, mientras que el valor de retorno (bool) indica si la operación fue exitosa. En las funciones de PE la cola se recibe por referencia (&) para trabajar sobre la cola original.

- Menú interactivo: el ciclo do-while repite el menú hasta que el usuario elige 0, y el switch ejecuta la operación elegida.

- Enfoque PE: la cola es una variable queue<int> y las operaciones (encolar, desencolar, primero, mostrar) son funciones globales que la reciben como parámetro.

- Enfoque POO: la clase ColaLibreria encapsula el objeto queue<int> como atributo privado y expone las operaciones como métodos públicos. No necesita constructor ni destructor propios, porque la librería se encarga de iniciar y liberar la cola.

- Lo que imprime: con la secuencia de ejemplo, se encolan 10, 20 y 30, se muestra la cola (10 20 30), se consulta el primero (10), se desencola el 10 y se muestra la cola final (20 30). El resultado es el mismo en PE y POO; solo cambia el título “Cola Dinamica con Libreria PE” / “Cola Dinamica con Libreria POO”.

- Errores y Soluciones: no hubo errores.

Ejercicio 10 - Manejo de una lista de manera dinámica por medio de las librerías (PE y POO)

Objetivo del código

Implementar una lista de manera dinámica utilizando la clase list de la biblioteca estándar de C++ (STL), en lugar de programar los nodos y punteros manualmente, aplicando los paradigmas de Programación Estructurada (PE) y Programación Orientada a Objetos (POO). El programa permite al usuario, mediante un menú en consola, insertar, eliminar, buscar y mostrar los datos de la lista en cualquier posición, controlando los casos de lista vacía y posición inválida.

Código PE

Ejercicio10_PE.cpp

```cpp
#include <iostream>
#include <list>
#include <algorithm>
#include <iterator>
using namespace std;

bool insertar(list<int> &l, int pos, int dato) {
    if (pos < 1 || pos > (int)l.size() + 1) return false;
    list<int>::iterator it = l.begin();
    advance(it, pos - 1);
    l.insert(it, dato);
    return true;
}

bool eliminar(list<int> &l, int pos, int &dato) {
    if (pos < 1 || pos > (int)l.size()) return false;
    list<int>::iterator it = l.begin();
    advance(it, pos - 1);
    dato = *it;
    l.erase(it);
    return true;
}

int buscar(list<int> &l, int dato) {
    list<int>::iterator it = find(l.begin(), l.end(), dato);
    if (it == l.end()) return 0;
    return distance(l.begin(), it) + 1;
}

void mostrar(list<int> &l) {
    if (l.empty()) {
        cout << "La lista esta vacia\n";
        return;
    }
    cout << "Lista: ";
    for (int x : l) cout << x << " ";
    cout << endl;
}

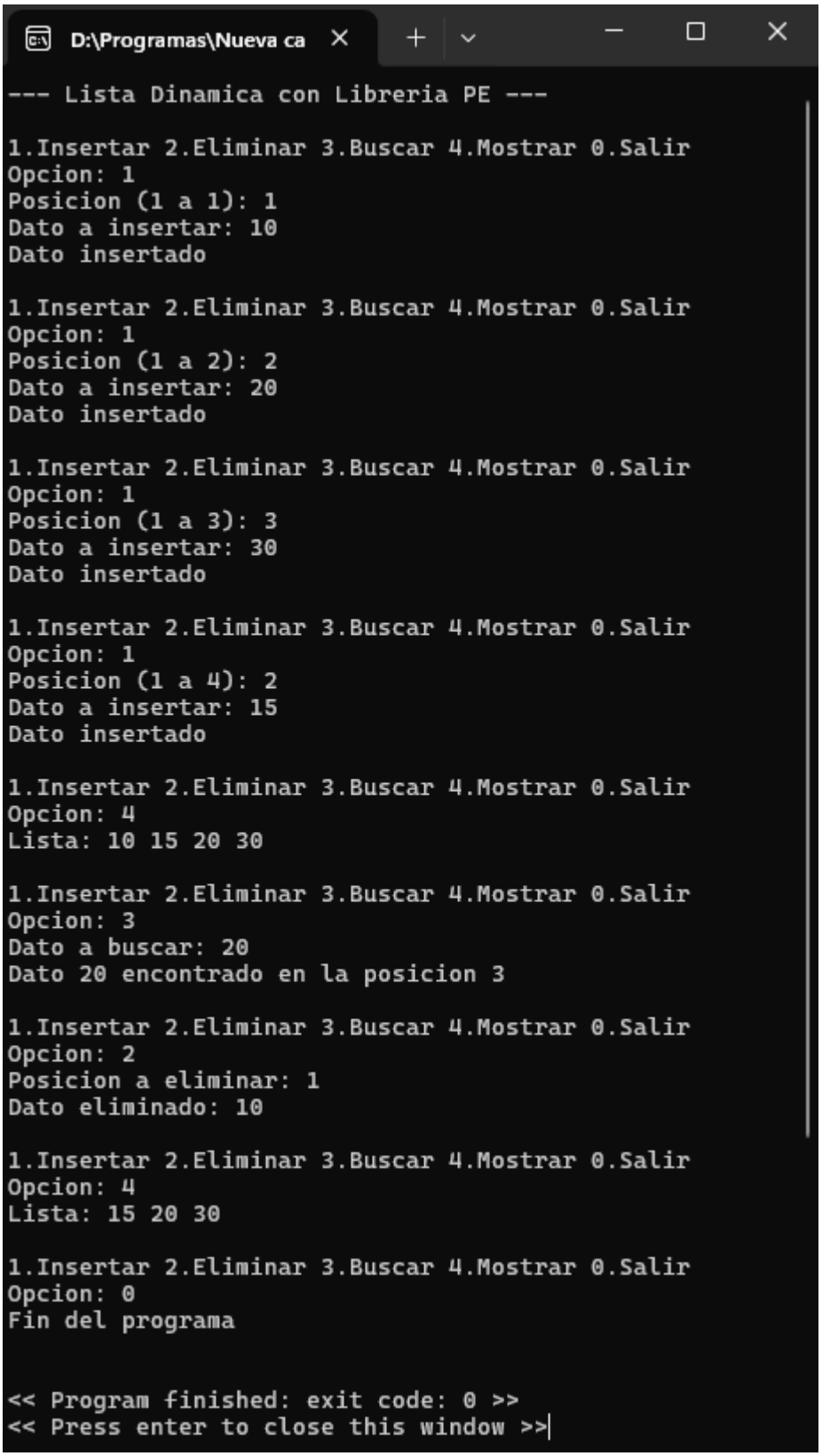
```

```cpp
int main() {
    list<int> lista;
    int opcion, dato, pos;

    cout << "--- Lista Dinamica con Libreria PE ---\n";
    do {
        cout << "\n1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Posicion (1 a " << lista.size() + 1 << "): ";
            cin >> pos;
            cout << "Dato a insertar: ";
            cin >> dato;
            if (insertar(lista, pos, dato)) cout << "Dato insertado\n";
            else cout << "Posicion invalida\n";
            break;
        case 2:
            cout << "Posicion a eliminar: ";
            cin >> pos;
            if (eliminar(lista, pos, dato)) cout << "Dato eliminado: " << dato << endl;
            else cout << "Lista vacia o posicion invalida\n";
            break;
        case 3:
            cout << "Dato a buscar: ";
            cin >> dato;
            pos = buscar(lista, dato);
            if (pos > 0) cout << "Dato " << dato << " encontrado en la posicion " << pos << endl;
            else cout << "Dato no encontrado\n";
            break;
        case 4:
            mostrar(lista);
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}

```

Salida


D:\Programas\Nueva ca
--- Lista Dinamica con Libreria PE ---
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 1): 1
Dato a insertar: 10
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 2): 2
Dato a insertar: 20
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 3): 3
Dato a insertar: 30
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 1
Posicion (1 a 4): 2
Dato a insertar: 15
Dato insertado
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 4
Lista: 10 15 20 30
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 3
Dato a buscar: 20
Dato 20 encontrado en la posicion 3
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 2
Posicion a eliminar: 1
Dato eliminado: 10
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 4
Lista: 15 20 30
1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir
Opcion: 0
Fin del programa
<< Program finished: exit code: 0 >>
<< Press enter to close this window >>

Código POO

Ejercicio10_POO.cpp

```cpp
#include <iostream>
#include <list>
#include <algorithm>
#include <iterator>
using namespace std;

class ListaLibreria {
private:
    list<int> lista;
public:
    int getCantidad() { return (int)lista.size(); }
    bool vacia()      { return lista.empty(); }

    bool insertar(int pos, int dato) {
        if (pos < 1 || pos > getCantidad() + 1) return false;
        list<int>::iterator it = lista.begin();
        advance(it, pos - 1);
        lista.insert(it, dato);
        return true;
    }

    bool eliminar(int pos, int &dato) {
        if (pos < 1 || pos > getCantidad()) return false;
        list<int>::iterator it = lista.begin();
        advance(it, pos - 1);
        dato = *it;
        lista.erase(it);
        return true;
    }

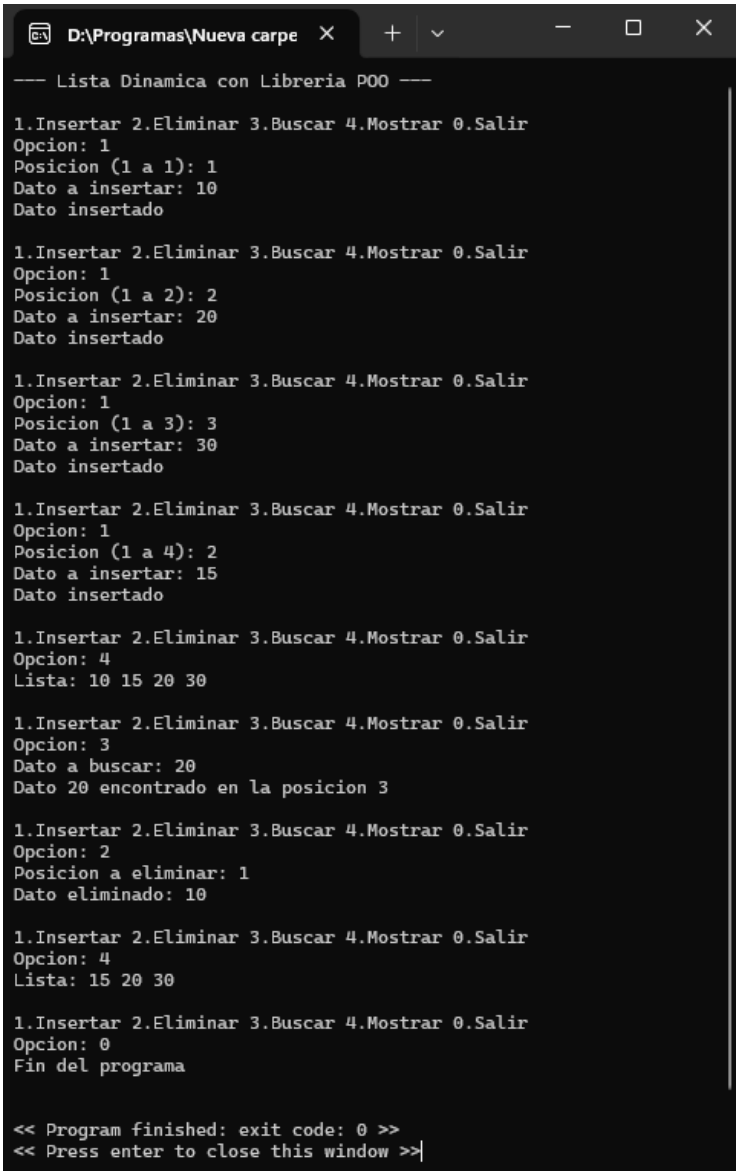
    int buscar(int dato) {
        list<int>::iterator it = find(lista.begin(), lista.end(), dato);
        if (it == lista.end()) return 0;
        return distance(lista.begin(), it) + 1;
    }

    void mostrar() {
        if (vacia()) {
            cout << "La lista esta vacia\n";
            return;
        }
        cout << "Lista: ";
        for (int x : lista) cout << x << " ";
        cout << endl;
    }
};

int main() {
    ListaLibreria lista;
    int opcion, dato, pos;

    cout << "--- Lista Dinamica con Libreria POO ---\n";
    do {
        cout << "\n1.Insertar 2.Eliminar 3.Buscar 4.Mostrar 0.Salir\nOpcion: ";
        cin >> opcion;
        switch (opcion) {
        case 1:
            cout << "Posicion (1 a " << lista.getCantidad() + 1 << "): ";
            cin >> pos;
            cout << "Dato a insertar: ";
            cin >> dato;
            if (lista.insertar(pos, dato)) cout << "Dato insertado\n";
            else cout << "Posicion invalida\n";
            break;
        case 2:
            cout << "Posicion a eliminar: ";
            cin >> pos;
            if (lista.eliminar(pos, dato)) cout << "Dato eliminado: " << dato << endl;
            else cout << "Lista vacia o posicion invalida\n";
            break;
        case 3:
            cout << "Dato a buscar: ";
            cin >> dato;
            pos = lista.buscar(dato);
            if (pos > 0) cout << "Dato " << dato << " encontrado en la posicion " << pos << endl;
            else cout << "Dato no encontrado\n";
            break;
        case 4:
            lista.mostrar();
            break;
        case 0:
            cout << "Fin del programa\n";
            break;
        default:
            cout << "Opcion invalida\n";
        }
    } while (opcion != 0);
    return 0;
}
```

Salida



Explicación del Código

- Lista con librería: en lugar de crear nodos y punteros manualmente, se utiliza la clase list de la biblioteca estándar (#include <list>). La clase es una lista doblemente enlazada que administra internamente la memoria de forma dinámica, por lo que la lista crece y se reduce según se necesite y no existe un tamaño máximo definido.

- Plantilla list<int>: el tipo entre los símbolos < > indica qué tipo de dato guarda la lista. Con list<int> la lista almacena números enteros.

- Posiciones: el usuario trabaja con posiciones desde 1, y el programa las valida con size() (de 1 a size() + 1 para insertar, y de 1 a size() para eliminar).

- Iteradores: un iterador (list<int>::iterator) funciona como un puntero que indica una posición dentro de la lista. Se inicia en begin() (primer elemento) y se mueve con la función advance(it, pos - 1) hasta la posición deseada. El dato al que apunta se obtiene con el operador de desreferencia (*it).

- Insertar: valida la posición, coloca un iterador en ella y usa insert(it, dato), que agrega el dato antes de esa posición y enlaza los nodos automáticamente. En el ejemplo, al insertar 15 en la posición 2, la lista queda 10 15 20 30.

- Eliminar: valida la posición, coloca un iterador en ella, guarda el dato con *it y lo elimina con erase(it). En el ejemplo, al eliminar la posición 1 se obtiene el 10 y la lista queda 15 20 30.

- Buscar: la función find() (#include <algorithm>) recorre la lista y regresa un iterador al dato buscado, o end() si no existe. La posición se calcula con distance(begin(), it) + 1, y se regresa 0 si el dato no se encontró.

- Mostrar: recorre la lista con un ciclo for basado en rango (for (int x : lista)), que visita cada elemento en orden sin necesidad de usar iteradores de forma explícita.

- Liberación de memoria: no es necesario usar new ni delete; la librería libera la memoria de los nodos automáticamente cuando la lista se destruye.

- Parámetros por referencia: en eliminar() el dato se devuelve por medio del parámetro &dato, mientras que el valor de retorno (bool) indica si la operación fue exitosa. En las funciones de PE la lista se recibe por referencia (&) para trabajar sobre la lista original.

- Menú interactivo: el ciclo do-while repite el menú hasta que el usuario elige 0, y el switch ejecuta la operación elegida.

- Enfoque PE: la lista es una variable list<int> y las operaciones (insertar, eliminar, buscar, mostrar) son funciones globales que la reciben como parámetro.

- Enfoque POO: la clase ListaLibreria encapsula el objeto list<int> como atributo privado y expone las operaciones como métodos públicos. No necesita constructor ni destructor propios, porque la librería se encarga de iniciar y liberar la lista.

- Lo que imprime: con la secuencia de ejemplo, se insertan 10, 20 y 30, se inserta 15 en la posición 2, se muestra la lista (10 15 20 30), se busca el 20 (posición 3), se elimina la posición 1 (dato 10) y se muestra la lista final (15 20 30). El resultado es el mismo en PE y POO; solo cambia el título “Lista Dinamica con Libreria PE” / “Lista Dinamica con Libreria POO”.

- Errores y Soluciones: no hubo errores.